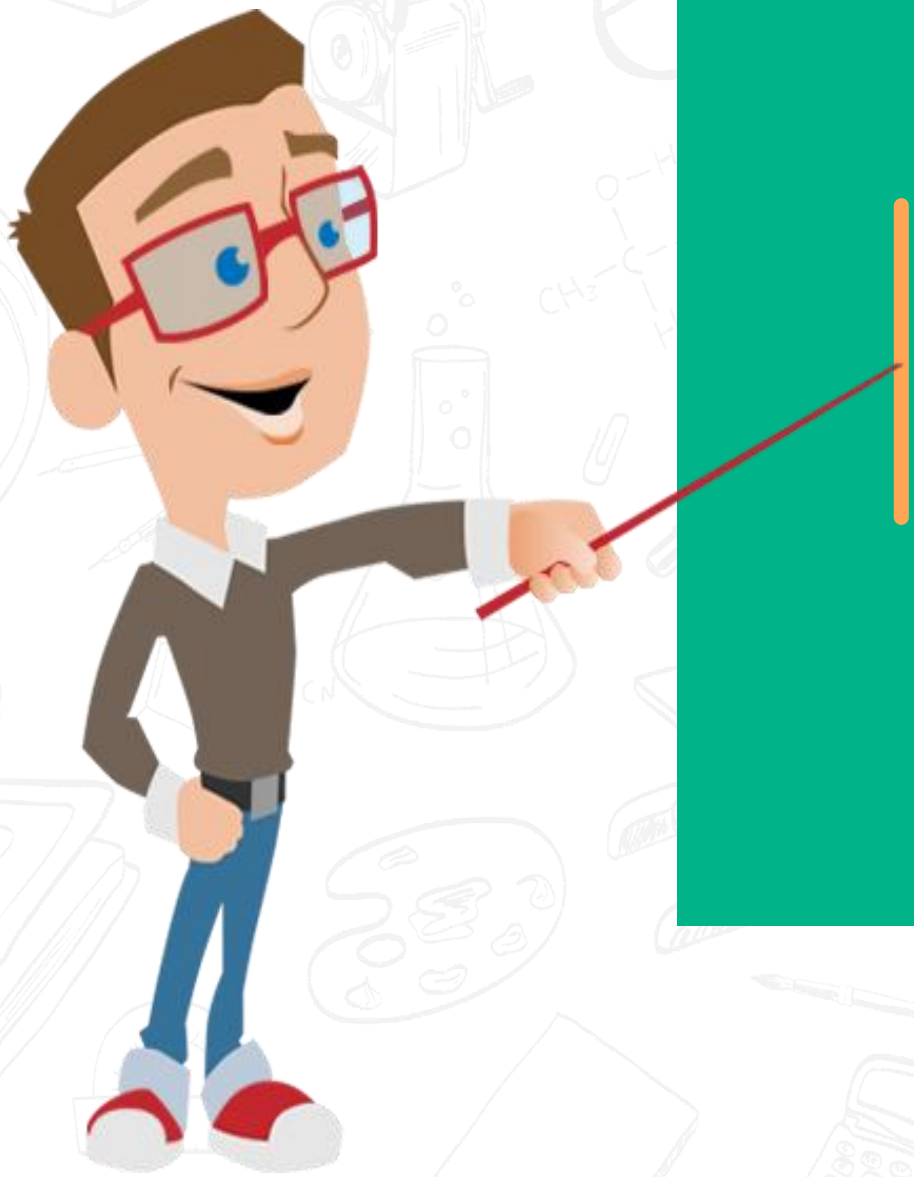


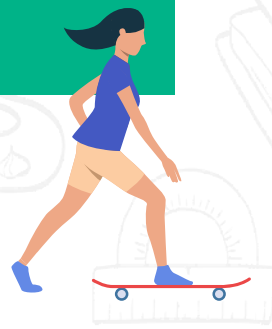
机器人操作系统（双语）

Robot Operating System (Bilingual)

WHERE THERE'S A WILL, THERE'S A WAY

授课教师：郄(qiè)龙飞

机器人工程



Overview lesson 12-1

1. 静态坐标变换
2. ~
3. ~
4. ~

Objectives

1、掌握静态坐标变换的 C++ 编程方法



Pre-assessment

1、是否用过TF坐标变换?

2、如何进行坐标变换的编程?



1

静态坐标变换

1.1 静态坐标变换

1.1.1 Overview

- 坐标变换 tf2

机器人系统通常具有许多随时间变化的三维坐标系，例如世界坐标系、基础坐标系、夹具坐标系、头部坐标系等。tf2随时间跟踪所有的这些坐标系，并可以解答下列问题：

① 在5秒之前，头部坐标系相对于世界坐标系在哪；

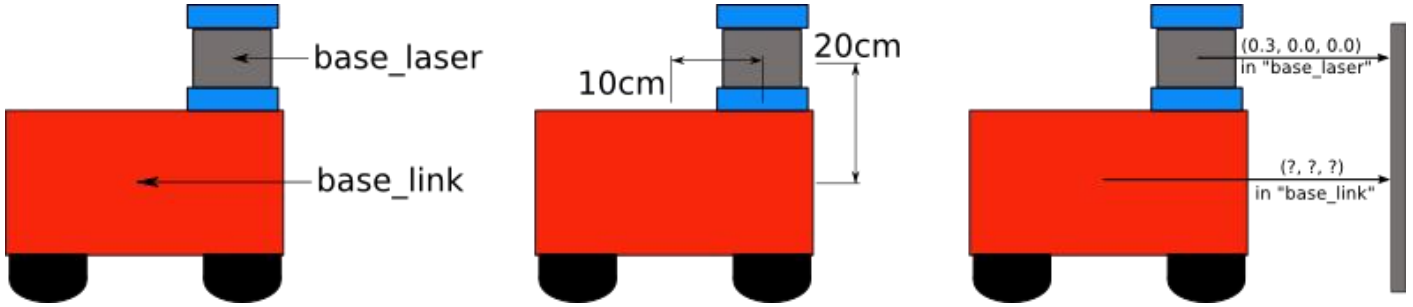
② 我抓手中的物体相对于我的底座的姿态是什么；

③ 地图坐标系中的基础坐标系当前位姿是什么。

1.1 静态坐标变换

1.1.2 Tf2 packages

- 静态坐标变换



- tf2 对应的常用功能包有：
  - tf2：tf2核心功能包，封装了坐标变换的常用消息
  - tf2_ros：为 tf2 提供了 roscpp 和 rospy 绑定，封装了坐标变换常用的 APIs
  - tf2_geometry_msgs：可以将 ROS 消息转换成 tf2 消息

1.1 静态坐标变换

1.1.2 Tf2 packages

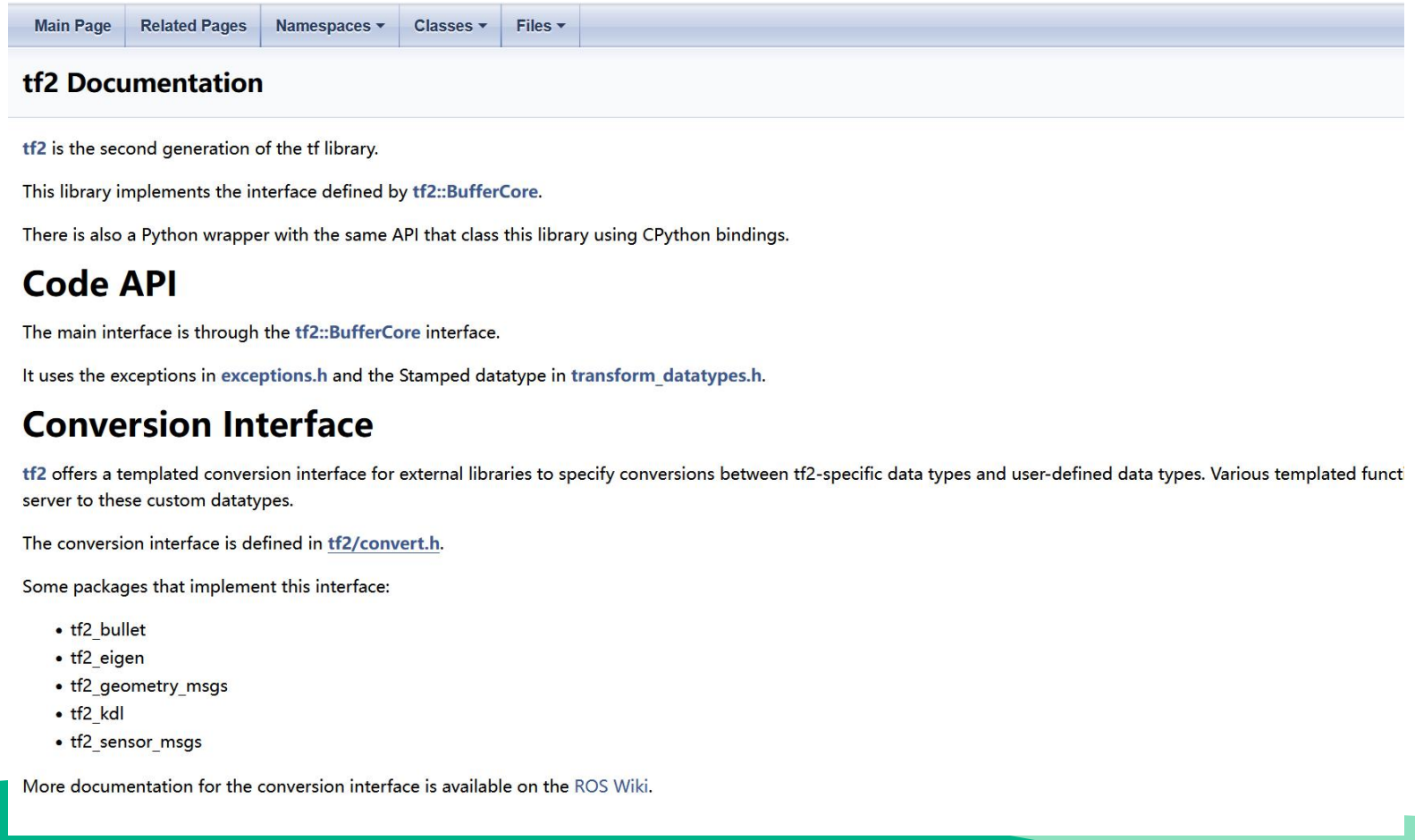
- **tf2:**
  - **tf2 功能包是 ROS 中 geometry2 元软件包下的一个重要软件包，是ROS的第二代机器人坐标变换库，允许用户随着时间的推移跟踪多个坐标系**



1.1 静态坐标变换

1.1.2 Tf2 packages

- tf2 APIs: http://docs.ros.org/en/noetic/api/tf2/html/



Main Page | Related Pages | Namespaces | Classes | Files

tf2 Documentation

tf2 is the second generation of the tf library.

This library implements the interface defined by tf2::BufferCore.

There is also a Python wrapper with the same API that class this library using CPython bindings.

Code API

The main interface is through the tf2::BufferCore interface.

It uses the exceptions in exceptions.h and the Stamped datatype in transform_datatypes.h.

Conversion Interface

tf2 offers a templated conversion interface for external libraries to specify conversions between tf2-specific data types and user-defined data types. Various templated funct server to these custom datatypes.

The conversion interface is defined in tf2/convert.h.

Some packages that implement this interface:

- tf2_bullet
- tf2_eigen
- tf2_geometry_msgs
- tf2_kdl
- tf2_sensor_msgs

More documentation for the conversion interface is available on the ROS Wiki.

1.1.2 Tf2 packages

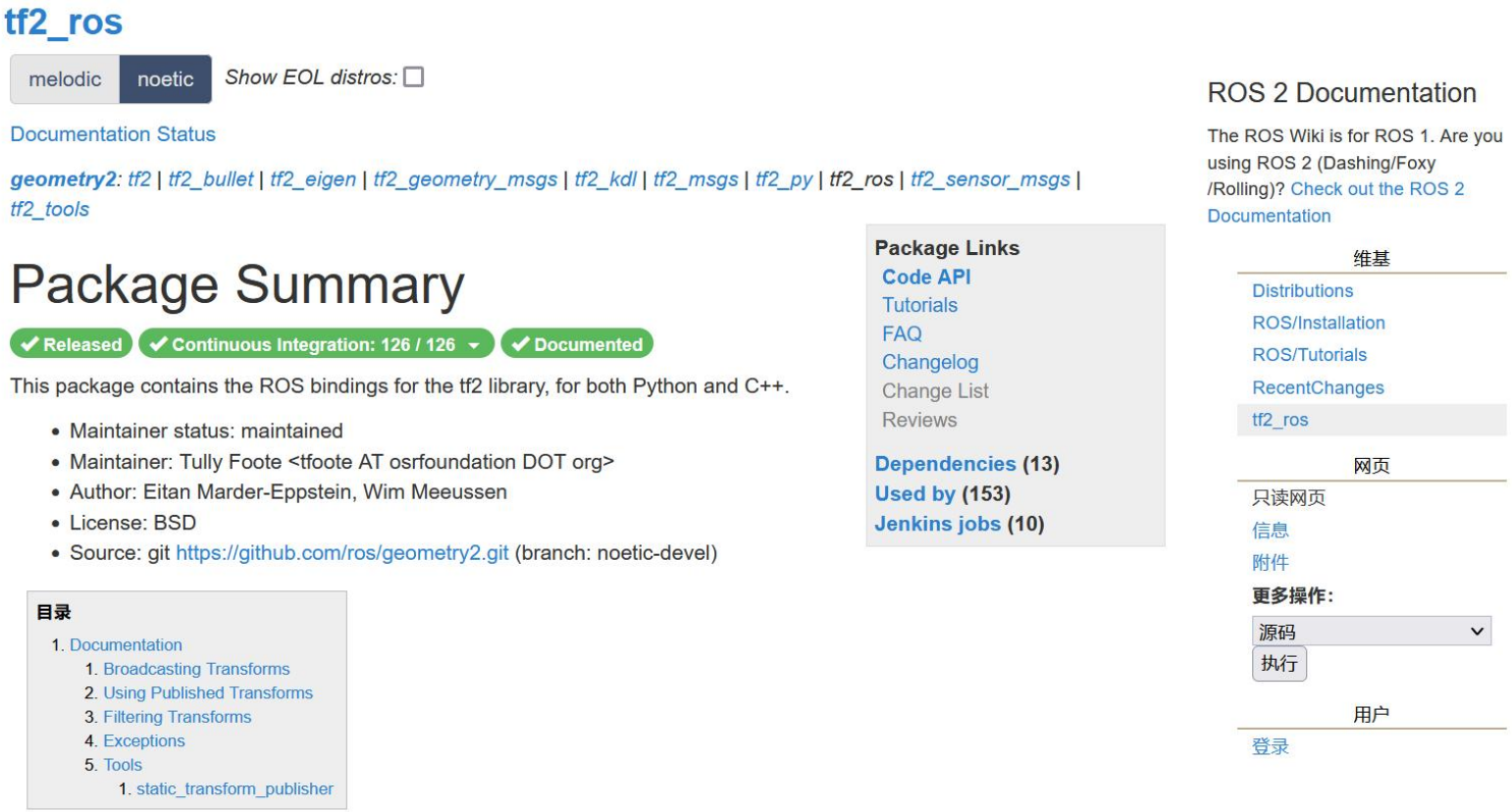
- **tf2:**
  - **tf2 功能包封装了坐标变换的常用消息，例如：**
    - **srd_msgs**
    - **geometry_msgs**
    - **tf2_msgs**

```
ros1@ros1-vb:~$ rospack depends tf2
cpp_common
rostime
roscpp_traits
roscpp_serialization
catkin
genmsg
genpy
message_runtime
std_msgs
geometry_msgs
gencpp
geneus
gennodejs
genlisp
message_generation
actionlib_msgs
tf2_msgs
```

1.1 静态坐标变换

1.1.2 Tf2 packages

- **tf2_ros:　此功能包包含 tf2 库的 ROS 绑定，适用于 Python 和 C++**



tf2_ros

melodic noetic *Show EOL distros:* ☐

Documentation Status

geometry2: *tf2* | *tf2_bullet* | *tf2_eigen* | *tf2_geometry_msgs* | *tf2_kdl* | *tf2_msgs* | *tf2_py* | *tf2_ros* | *tf2_sensor_msgs* | *tf2_tools*

Package Summary

Released | Continuous Integration: 126 / 126 | Documented

This package contains the ROS bindings for the tf2 library, for both Python and C++.

- Maintainer status: maintained
- Maintainer: Tully Foote <tfoote AT osrfoundation DOT org>
- Author: Eitan Marder-Eppstein, Wim Meeussen
- License: BSD
- Source: git https://github.com/ros/geometry2.git (branch: noetic-devel)

目录

1. Documentation
   1. Broadcasting Transforms
   2. Using Published Transforms
   3. Filtering Transforms
   4. Exceptions
   5. Tools
      1. static_transform_publisher

Package Links
Code API
Tutorials
FAQ
Changelog
Change List
Reviews

Dependencies (13)
Used by (153)
Jenkins jobs (10)

ROS 2 Documentation

The ROS Wiki is for ROS 1. Are you using ROS 2 (Dashing/Foxy /Rolling)? Check out the ROS 2 Documentation

维基
Distributions
ROS/Installation
ROS/Tutorials
RecentChanges
tf2_ros

网页
只读网页
信息
附件
更多操作:
源码
执行

用户
登录

http://wiki.ros.org/tf2_ros?distro=melodic

1.1 静态坐标变换

1.1.2 Tf2 packages

- **tf2_ros APIs** **http://docs.ros.org/en/noetic/api/tf2_ros/html/c++/**

To broadcast transforms using ROS:

1. Call ros::init() to initialize a node.
2. Construct a **tf2_ros::TransformBroadcaster**.
3. Pass a geometry_msgs::TransformStamped message to **tf2_ros::TransformBroadcaster::sendTransform()**.
   - Alternatively, pass a vector of geometry_msgs::TransformStamped messages.
- Use **tf2_ros::StaticTransformBroadcaster** for "latching" behavior when transforms that are not expected to change.

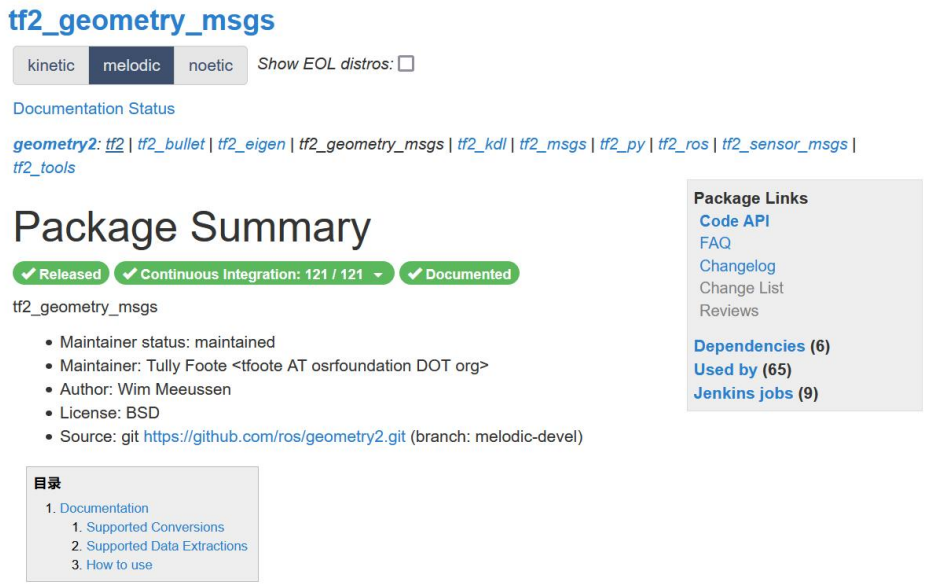
To listen for transforms using ROS:

- Construct an instance of a class that implements **tf2_ros::BufferInterface**.
  1. **tf2_ros::Buffer** is the standard implementation which offers a tf2_frames service that can respond to requests with a tf2_msgs::FrameGraph.
  - **tf2_ros::BufferClient** uses an actionlib::SimpleActionClient to wait for the requested transform to become available.
    - It should be used with a **tf2_ros::BufferServer**, which offers the corresponding actionlib::ActionSErver.
2. Pass the **tf2_ros::Buffer** to the constructor of **tf2_ros::TransformListener**.
   - Optionally, pass a ros::NodeHandle (otherwise TransformListener will connect to the node for the process).
   - Optionally, specify if the TransformListener runs in its own thread or not.
3. Use **tf2_ros::BufferInterface::transform()** to apply a transform on the tf server to an input frame.
   - Or, check if a transform is available with **tf2_ros::BufferInterface::canTransform()**.
   - Then, call **tf2_ros::BufferInterface::lookupTransform()** to get the transform between two frames.
- Construct a **tf2_ros::MessageFilter** with the TransformListener to apply a transformation to incoming frames.
  - This is especially useful when streaming sensor data.

1.1 静态坐标变换

1.1.2 Tf2 packages

- **tf2_geometry_msgs:**
  - **tf2_geometry_msgs 包含用于在各种 geometry_msgs 数据类型之间进行转换的函数**



tf2_geometry_msgs

kinetic | melodic | noetic Show EOL distros:

Documentation Status

geometry2: tf2 | tf2_bullet | tf2_eigen | tf2_geometry_msgs | tf2_kdl | tf2_msgs | tf2_py | tf2_ros | tf2_sensor_msgs | tf2_tools

Package Summary

Released | Continuous Integration: 121 / 121 | Documented

tf2_geometry_msgs

- Maintainer status: maintained
- Maintainer: Tully Foote <tfoote AT osrfoundation DOT org>
- Author: Wim Meeussen
- License: BSD
- Source: git https://github.com/ros/geometry2.git (branch: melodic-devel)

Package Links
Code API
FAQ
Changelog
Change List
Reviews

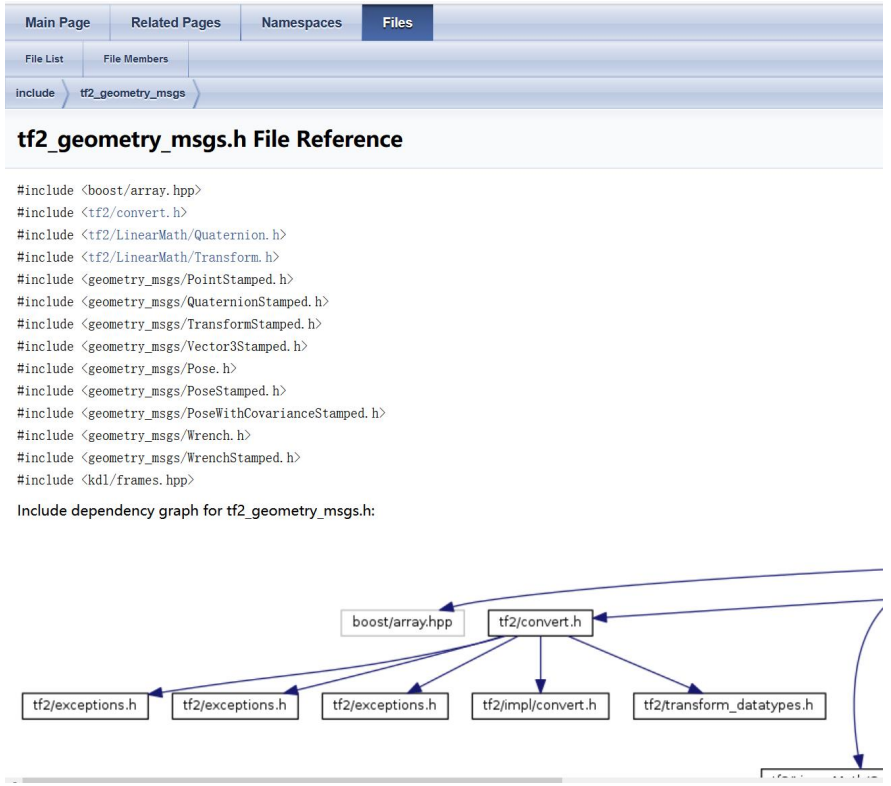
Dependencies (6)
Used by (65)
Jenkins jobs (9)

目录
1. Documentation
   1. Supported Conversions
   2. Supported Data Extractions
   3. How to use

1. Documentation

This package allows to convert ros messages to tf2 messages and to retrieve data from ros messages.

1.1 Supported Conversions



Main Page | Related Pages | Namespaces | Files

File List | File Members

include > tf2_geometry_msgs

tf2_geometry_msgs.h File Reference

```
#include <boost/array.hpp>
#include <tf2/convert.h>
#include <tf2/LinearMath/Quaternion.h>
#include <tf2/LinearMath/Transform.h>
#include <geometry_msgs/PointStamped.h>
#include <geometry_msgs/QuaternionStamped.h>
#include <geometry_msgs/TransformStamped.h>
#include <geometry_msgs/Vector3Stamped.h>
#include <geometry_msgs/Pose.h>
#include <geometry_msgs/PoseStamped.h>
#include <geometry_msgs/PoseWithCovarianceStamped.h>
#include <geometry_msgs/Wrench.h>
#include <geometry_msgs/WrenchStamped.h>
#include <kdl/frames.hpp>
```

Include dependency graph for tf2_geometry_msgs.h:

http://wiki.ros.org/tf2_geometry_msgs?distro=melodic

1.1 静态坐标变换

1.1.2 Tf2 packages

- tf2_geometry_msgs APIs: http://docs.ros.org/en/noetic/api/tf2_geometry_msgs/html/c++/

Main Page | Related Pages | Namespaces | Files

tf2_geometry_msgs Documentation

tf2_geometry_msgs contains functions for converting between various geometry_msgs data types.

This library is an implementation of the templated conversion interface specified in tf/convert.h. It offers conversion and transform convenience functions
data types, such as Vector, Point, Pose, Transform, Quaternion, etc.

See the Conversions overview wiki page for more information about datatype conversion in **tf2**.

Code API

This library consists of one header only, **tf2_geometry_msgs/tf2_geometry_msgs.h**, which consists mostly of specializations of template functions define

1.1.3 Tf2 messages

- 坐标转换实现中常用的消息类型
  - geometry_msgs/TransformStamped
  - geometry_msgs/PointStamped
- 前者用于传输坐标系相关位置信息，后者用于传输某个坐标系内坐标点的信息
- 在坐标变换中，频繁的使用到坐标系的相对关系以及坐标点信息

1.1 静态坐标变换

1.1.3 Tf2 messages

- geometry_msgs/TransformStamped

```
$ rosmsg info geometry_msgs/TransformStamped
```

```
①  std_msgs/Header header                    #头信息
      uint32 seq                              #|-- 序列号
      time stamp                              #|-- 时间戳
      string frame_id                          #|-- 坐标 ID
②  string child_frame_id                    #子坐标系的 id
③  geometry_msgs/Transform transform        #坐标信息
      geometry_msgs/Vector3 translation       #偏移量
        float64 x                              #|-- X 方向的偏移量
        float64 y                              #|-- Y 方向的偏移量
        float64 z                              #|-- Z 方向上的偏移量
      geometry_msgs/Quaternion rotation       #四元数
        float64 x
        float64 y
        float64 z
        float64 w
```

1.1 静态坐标变换

1.1.3 Tf2 messages

- geometry_msgs/PointStamped

```
$ rosmsg info geometry_msgs/PointStamped
```

```
①  std_msgs/Header header                    #头
      uint32 seq                               #|-- 序号
      time stamp                               #|-- 时间戳
      string frame_id                          #|-- 所属坐标系的 id
②  geometry_msgs/Point point                 #点坐标
      float64 x                                #|-- x y z 坐标
      float64 y
      float64 z
```

1.1.4 Time and duration

- ROS 里经常用到时间变量，比如坐标变换、基于时间和控制量计算机器人的移动距离、设定程序的等待时间/循环时间、设定计时器等
- roscpp 里有两种时间表示方法：时刻 (ros::Time) 和时长(ros::Duration)。其中Duration可以是负数
- Time和Duration拥有一样的成员：

  int32 sec

  int32 nsec

  sec是秒，nsec是纳秒

参考资料：https://blog.csdn.net/u014453443/article/details/104636566

1.1.4 Time and duration

- 使用 Time 和 Duration，需要分别 #include<ros/time.h> 和 #include <ros/duration.h>

```
//定义类对象
ros::Time::Time(uint32_t _sec, uint32_t _nsec)
ros::Time::Time(double t)

ros::Duration::Duration(uint32_t _sec, uint32_t _nsec)
ros::Duration::Duration(double t)

//故ros::Time a_little_after_the_beginning(0, 1000000);  等价于ros::Time
a_little_after_the_beginning(0.001);
```

1.1.4 Time and duration

- 使用 Time 和 Duration，需要分别 #include<ros/time.h> 和 #include <ros/duration.h>

```
ros::Time begin=ros::Time::now();          //获取当前时间
ros::Time double1=ros::Time();             //获取0时刻时间，默认值为0
ros::Time double2=ros::Time(1);            //获取1时刻时间，默认值为0
ros::Time at_some_time1(5,20000000);       //逗号之前表示 second，逗号之后表示 nanosecond
ros::Time at_some_time2(5.2);              //同上,重载了float类型和两个uint类型的构造函数
ros::Duration one_hour(60*60,0);           //1h
double secs1=at_some_time1.toSec();        //将 Time 转为 double 型时间
double secs2=one_hour.toSec();             //将 Duration 转为 double 型时间
std::cout << "Test1:"
<< "\nbegin:" << begin
<< "\ndouble1:" << double1
<< "\ndouble2:" << double2
<< "\nat_some_time1:" << at_some_time1
<< "\nat_some_time2:" << at_some_time2
<< "\none_hour:" << one_hour
<< "\nsecs1:" << secs1
<< "\nsecs2:" << secs2
<< std::endl;
```

上述代码在ros_time.cpp 文件中

1.1.4 Time and duration

- 使用 Time 和 Duration进行加减运算：

```
ros::Time t1=ros::Time::now()-ros::Duration(5.5);     //t1是5.5s前的时刻,Time加减Duration返回Time
ros::Time t2=ros::Time::now()+ros::Duration(3.3);     //t2是当前时刻往后推3.3s的时刻
ros::Duration d1=t2-t1;                               //从t1到t2的时长，两个Time相减返回Duration类型
ros::Duration d2=d1-ros::Duration(0,300);             //两个Duration相减，返回Duration
std::cout << "\nTest2:"
<< "\nt1:" << t1
<< "\nt2:" << t2
<< "\nd1:" << d1
<< "\nd2:" << d2
<< std::endl;
```

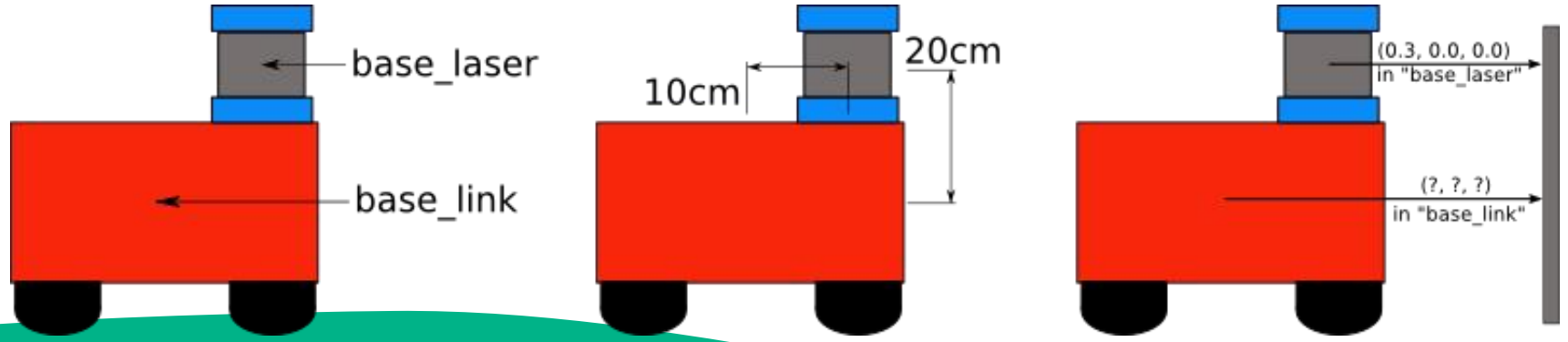
注意：没有Time+Time的运算

上述代码在ros_time.cpp 文件中

1.1 静态坐标变换

1.1.5 C++ code

- **需求描述:**
  - 现有一机器人模型，核心构成包含主体与雷达，各对应一坐标系，坐标系的原点分别位于主体与雷达的物理中心
  - 已知雷达原点相对于主体原点位移关系如下: x=0.1, y=0.0, z=0.2。当前雷达检测到一障碍物，在雷达坐标系中障碍物的坐标为 (2.0 3.0 5.0),请问，该障碍物相对于主体的坐标是多少?



1.1.5 C++ code

- **功能实现分析**：

  1. 发布方发布坐标系相对位置关系
  2. 订阅方订阅到发布的坐标系相对位置关系，再传入坐标点信息，然后借助于 tf 实现坐标变换，并将结果输出

- **整体实现流程**：

  1. 新建功能包，添加依赖
  2. 编写发布方实现
  3. 编写订阅方实现
  4. 执行并查看结果

1.1 静态坐标变换

1.1.5 C++ code

- **发布方实现流程：**

  1.包含头文件

  2.初始化 ROS 节点

  3.创建静态坐标转换广播器

  4.创建坐标系信息

  5.广播器发布坐标系信息，进行坐标系之间的变换

  6.spin()

1.1 静态坐标变换

1.1.5 C++ code

- 创建功能包
  - 项目功能包依赖为： tf2 tf2_ros tf2_geometry_msgs roscpp rospy std_msgs geometry_msgs

- 创建静态坐标变换广播器：

```
#include "tf2_ros/static_transform_broadcaster.h"        // 包含广播器头文件
tf2_ros::StaticTransformBroadcaster broadcaster          // 实例化广播器对象
```

To broadcast transforms using ROS:

1. Call ros::init() to initialize a node.
2. Construct a tf2_ros::TransformBroadcaster.
3. Pass a geometry_msgs::TransformStamped message to tf2_ros::TransformBroadcaster::sendTransform().
   - Alternatively, pass a vector of geometry_msgs::TransformStamped messages.
- Use tf2_ros::StaticTransformBroadcaster for "latching" behavior when transforms that are not expected to change.

1.1 静态坐标变换

1.1.5 C++ code

- 发布方(Bilibili)：

```
#include "ros/ros.h"
#include "tf2_ros/static_transform_broadcaster.h"
#include "geometry_msgs/TransformStamped.h"
#include "tf2/LinearMath/Quaternion.h"

int main(int argc, char *argv[])
{
    setlocale(LC_ALL,"");
    ros::init(argc,argv,"tf_static_pub");
    tf2_ros::StaticTransformBroadcaster broadcaster;
    geometry_msgs::TransformStamped ts;
    ts.header.seq = 100;
    ts.header.stamp = ros::Time::now();
    ts.header.frame_id = "baselink";
    ts.child_frame_id = "laser";
    ts.transform.translation.x = 0.1;
    ts.transform.translation.y = 0.0;
    ts.transform.translation.z = 0.2;
```

上述代码在tf_static_pub.cpp 文件中

1.1.5 C++ code

- 发布方(Bilibili)：

```
    tf2::Quaternion qtn;
    qtn.setRPY(0,0,0);
    ts.transform.rotation.x = qtn.getX();
    ts.transform.rotation.y = qtn.getY();
    ts.transform.rotation.z = qtn.getZ();
    ts.transform.rotation.w = qtn.getW();
     broadcaster.sendTransform(ts);
    ros::spin();
    return 0;
}
```

上述代码在tf_static_pub.cpp 文件中 代码详解：https://www.bilibili.com/video/BV1Ci4y1L7ZZ?p=185

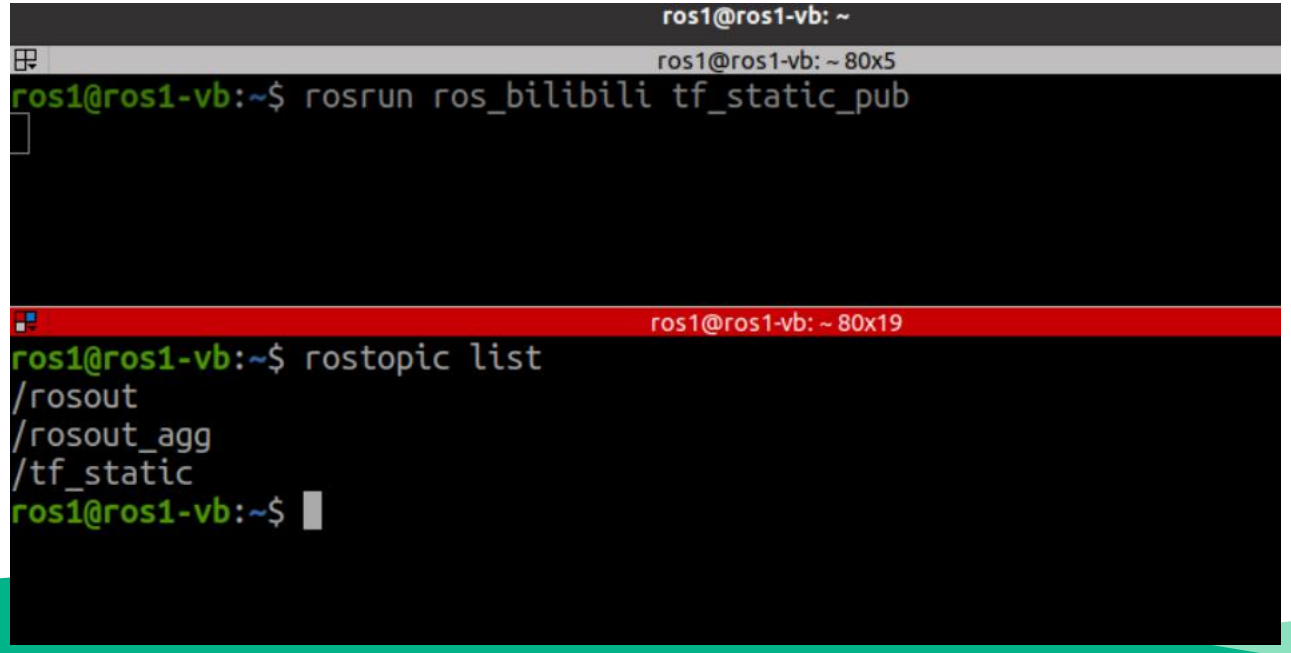
1.1 静态坐标变换

1.1.5 C++ code

- 发布方(Bilibili)：

```
$ rosrun ros_bilibili tf_static_pub
```

```
$ rostopic list
```



1.1 静态坐标变换

1.1.5 C++ code

- **发布方(Bilibili)：**

tf2_ros Documentation

tf2_ros is the C++ ROS wrapper around the tf2 transform library.

Code API

To broadcast transforms using ROS:

- Call ros::init() to initialize a node.
- Construct a **tf2_ros::TransformBroadcaster**.
- Pass a geometry_msgs::TransformStamped message to **tf2_ros::TransformBroadcaster::sendTransform()**.
  - Alternatively, pass a vector of geometry_msgs::TransformStamped messages.
- Use **tf2_ros::StaticTransformBroadcaster** for "latching" behavior when transforms that are not expected to change.

To listen for transforms using ROS:

- Construct an instance of a class that implements **tf2_ros::BufferInterface**.
  - **tf2_ros::Buffer** is the standard implementation which offers a tf2_frames service that can respond to requests with a tf2_msgs::FrameGrap
  - **tf2_ros::BufferClient** uses an actionlib::SimpleActionClient to wait for the requested transform to become available.
    - It should be used with a **tf2_ros::BufferServer**, which offers the corresponding actionlib::ActionSErver.
- Pass the **tf2_ros::Buffer** to the constructor of **tf2_ros::TransformListener**.
  - Optionally, pass a ros::NodeHandle (otherwise TransformListener will connect to the node for the process).
  - Optionally, specify if the TransformListener runs in its own thread or not.

1.1 静态坐标变换

1.1.5 C++ code

- **发布方(Bilibili)：**

tf2_ros::StaticTransformBroadcaster Class Reference

This class provides an easy way to publish coordinate frame transform information.
It will handle all the messaging and stuffing of messages. And the function prototypes lay out all the necessary data needed for each message.
More...

`#include <static_transform_broadcaster.h>`

Public Member Functions

void **sendTransform** (const geometry_msgs::TransformStamped &transform)
Send a TransformStamped message The stamped data structure includes frame_id, and time, and parent_id already.
More...

void **sendTransform** (const std::vector< geometry_msgs::TransformStamped > &transforms)
Send a vector of TransformStamped messages The stamped data structure includes frame_id, and time, and parent_id already.
More...

StaticTransformBroadcaster ()
Constructor (needs a ros::Node reference) More...

Private Attributes

tf2_msgs::TFMessage **net_message_**

ros::NodeHandle **node_**

1.1 静态坐标变换

1.1.5 C++ code

- **发布方(Bilibili)：**

sendTransform() [1/2]

void tf2_ros::StaticTransformBroadcaster::sendTransform (const geometry_msgs::TransformStamped & transform)

Send a TransformStamped message The stamped data structure includes frame_id, and time, and parent_id already.

Definition at line 57 of file static_transform_broadcaster.h.

sendTransform() [2/2]

void tf2_ros::StaticTransformBroadcaster::sendTransform (const std::vector< geometry_msgs::TransformStamped > & transforms)

Send a vector of TransformStamped messages The stamped data structure includes frame_id, and time, and parent_id already.

Definition at line 46 of file static_transform_broadcaster.cpp.

1.1 静态坐标变换

1.1.5 C++ code

- **发布方(Bilibili)：**

```
29
30
34  #include "ros/ros.h"
35  #include "tf2_msgs/TFMessage.h"
36  #include "tf2_ros/static_transform_broadcaster.h"
37  #include <algorithm>
38
39  namespace tf2_ros {
40
41  StaticTransformBroadcaster::StaticTransformBroadcaster()
42  {
43    publisher_ = node_.advertise<tf2_msgs::TFMessage>("/tf_static", 100, true);
44  }
45
46  void StaticTransformBroadcaster::sendTransform(const std::vector<geometry_msgs::TransformStamped> & msgtf)
47  {
48    for (const geometry_msgs::TransformStamped& input : msgtf)
49    {
50      auto predicate = [&input](const geometry_msgs::TransformStamped existing) {
51        return input.child_frame_id == existing.child_frame_id;
52      };
53      auto existing = std::find_if(net_message_.transforms.begin(), net_message_.transforms.end(), predicate);
54
55      if (existing != net_message_.transforms.end())
56        *existing = input;
57      else
58        net_message_.transforms.push_back(input);
59    }
60
61    publisher_.publish(net_message_);
62  }
63
64  }
```

1.1 静态坐标变换

1.1.5 C++ code

- 发布方(Bilibili)：

```
$ rostopic echo /tf_static
```

```
ros1@ros1-vb:~$ rostopic echo /tf_static
transforms:
  -
    header:
      seq: 100
      stamp:
        secs: 1669439192
        nsecs: 729016126
      frame_id: "base_link"
    child_frame_id: "laser"
    transform:
      translation:
        x: 0.1
        y: 0.0
        z: 0.2
      rotation:
        x: 0.0
        y: 0.0
        z: 0.0
        w: 1.0
---
```

1.1 静态坐标变换

1.1.5 C++ code

- **订阅方实现流程：**

  1.包含头文件

  2.初始化 ROS 节点

  3.创建 TF 静态坐标监听器

  4.生成一个坐标点(相对于子级坐标系)，

  5.转换坐标点(相对于父级坐标系)，进行坐标点和基坐标系的变换

  6.spinOnce()

1.1 静态坐标变换

1.1.5 C++ code

- 创建静态坐标变换监听器：

```
#include "tf2_ros/buffer.h"                    // 包含缓存头文件
#include "tf2_ros/transform_listener.h"        // 包含监听器头文件
tf2_ros::Buffer buffer;                        // 实例化缓存对象
tf2_ros::TransformListener listener(buffer);   // 实例化监听对象
```

To listen for transforms using ROS:

- Construct an instance of a class that implements **tf2_ros::BufferInterface**.

1
  - **tf2_ros::Buffer** is the standard implementation which offers a tf2_frames service that can respond to requests with a tf2_msgs::FrameGraph.
  - **tf2_ros::BufferClient** uses an actionlib::SimpleActionClient to wait for the requested transform to become available.
    - It should be used with a **tf2_ros::BufferServer**, which offers the corresponding actionlib::ActionSErver.

2
- Pass the **tf2_ros::Buffer** to the constructor of **tf2_ros::TransformListener**.
  - Optionally, pass a ros::NodeHandle (otherwise TransformListener will connect to the node for the process).
  - Optionally, specify if the TransformListener runs in its own thread or not.

3
- Use **tf2_ros::BufferInterface::transform()** to apply a transform on the tf server to an input frame.
  - Or, check if a transform is available with **tf2_ros::BufferInterface::canTransform()**.
  - Then, call **tf2_ros::BufferInterface::lookupTransform()** to get the transform between two frames.
- Construct a **tf2_ros::MessageFilter** with the TransformListener to apply a transformation to incoming frames.
  - This is especially useful when streaming sensor data.

1.1.5 C++ code

- 订阅方(Bilibili)：

```
#include "ros/ros.h"
#include "tf2_ros/transform_listener.h"
#include "tf2_ros/buffer.h"
#include "geometry_msgs/PointStamped.h"
#include "tf2_geometry_msgs/tf2_geometry_msgs.h"

int main(int argc, char *argv[])
{
    setlocale(LC_ALL,"");
    ros::init(argc,argv,"tf_static_sub");
    ros::NodeHandle nh;
    tf2_ros::Buffer buffer;
    tf2_ros::TransformListener listener(buffer);

    ros::Rate r(1);
```

上述代码在tf_static_sub.cpp 文件中

1.1.5 C++ code

- 订阅方(Bilibili):

```
while (ros::ok())
{
    geometry_msgs::PointStamped point_laser;
    point_laser.header.frame_id = "laser";
    point_laser.header.stamp = ros::Time::now();
    point_laser.point.x = 2;
    point_laser.point.y = 3;
    point_laser.point.z = 5;
    try
    {
        geometry_msgs::PointStamped point_base;
        point_base = buffer.transform(point_laser,"baselink");
        ROS_INFO("转换后的数据:(%.2f,%.2f,%.2f),参考的坐标系是:(%s)",
point_base.point.x,point_base.point.y,point_base.point.z,point_base.header.frame_id.c_str());
    }
```

上述代码在tf_static_sub.cpp 文件中

1.1.5 C++ code

- 订阅方(Bilibili)：

```
        catch(const std::exception& e)
        {
            // std::cerr << e.what() << '\n';
            ROS_INFO("程序异常.....");
        }

        r.sleep();
        ros::spinOnce();
    }

    return 0;
}
```

上述代码在tf_static_sub.cpp 文件中

1.1.6 Supplemental information

- 发布方(ROS_wiki)：

```
#include <ros/ros.h>
#include <tf2_ros/static_transform_broadcaster.h>      // 静态坐标变换
#include <geometry_msgs/TransformStamped.h>            // 静态坐标变换消息类型
#include <cstdio>                                      // C++输入输出标准库，cstdio中的函数定义在名称空间std里
#include <tf2/LinearMath/Quaternion.h>                 // 四元数计算


std::string static_turtle_name;
int main(int argc, char **argv)
{
  ros::init(argc,argv, "my_static_tf2_broadcaster");
  if(argc != 8)
  {
    ROS_ERROR("Invalid number of parameters\nusage: static_turtle_tf2_broadcaster child_frame_name x y z roll pitch yaw");
    return -1;
  }
  if(strcmp(argv[1],"world")==0)
  {
    ROS_ERROR("Your static turtle name cannot be 'world'");
    return -1;
  }
```

上述代码在static_turtle_tf2_broadcaster.cpp 文件中

1.1.6 Supplemental information

- 发布方(ROS_wiki)：

```
static_turtle_name = argv[1];
static tf2_ros::StaticTransformBroadcaster static_broadcaster;
geometry_msgs::TransformStamped static_transformStamped;

static_transformStamped.header.seq = 1;
static_transformStamped.header.stamp = ros::Time::now();
static_transformStamped.header.frame_id = "world";
static_transformStamped.child_frame_id = static_turtle_name;
static_transformStamped.transform.translation.x = atof(argv[2]);
static_transformStamped.transform.translation.y = atof(argv[3]);
static_transformStamped.transform.translation.z = atof(argv[4]);
tf2::Quaternion quat;
quat.setRPY(atof(argv[5]), atof(argv[6]), atof(argv[7]));
static_transformStamped.transform.rotation.x = quat.x();
static_transformStamped.transform.rotation.y = quat.y();
static_transformStamped.transform.rotation.z = quat.z();
static_transformStamped.transform.rotation.w = quat.w();
static_broadcaster.sendTransform(static_transformStamped);
ROS_INFO("Spinning until killed publishing %s to world", static_turtle_name.c_str());
ros::spin();
return 0;
};
```

上述代码在static_turtle_tf2_broadcaster.cpp 文件中

1.1.6 Supplemental information

- 发布方(ROS_wiki)：

```
$ rosrun tf2_roswiki static_turtle_tf2_broadcaster "laser" 0.1 0 0.2 0 0 0
```

```
ros1@ros1-vb:~$ rosrun tf2_roswiki static_turtle_tf2_broadcaster "laser" 0.1 0 0.2 0 0 0
[ INFO] [1669442046.250250893]: Spinning until killed publishing laser to world
```

```
$ rosrun ros_bilibili tf_static_sub
```

```
^Cros1@ros1-vb:~$ rosrun ros_bilibili tf_static_sub
[ INFO] [1669442086.863826630]: 程序异常.....
[ INFO] [1669442087.864872745]: 程序异常.....
[ INFO] [1669442088.872723260]: 程序异常.....
[ INFO] [1669442089.863831294]: 程序异常.....
[ INFO] [1669442090.868856455]: 程序异常.....
[ INFO] [1669442091.863839119]: 程序异常.....
[ INFO] [1669442092.865250980]: 程序异常.....
[ INFO] [1669442093.866170087]: 程序异常.....
```

上述代码在static_turtle_tf2_broadcaster.cpp 文件中

1.1.6 Supplemental information

- 发布方(ROS_wiki)：

tf_static_pub.cpp | static_turtle_tf2_broadcaster.cpp | tf_static_sub.cpp | CMakeLists.txt

tf2_roswiki > src > static_turtle_tf2_broadcaster.cpp > main(int, char **)

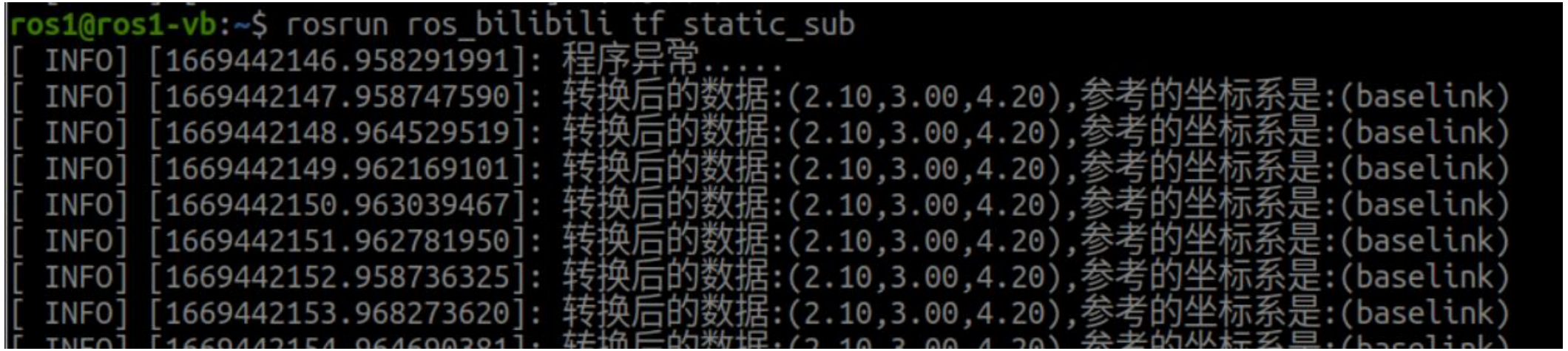
```
20      ROS_ERROR("Your static turtle name cannot be 'world'");
21      return -1;
22
23    }
24    static_turtle_name = argv[1];
25    static tf2_ros::StaticTransformBroadcaster static_broadcaster;
26    geometry_msgs::TransformStamped static_transformStamped;
27
28    static_transformStamped.header.seq = 1;
29    static_transformStamped.header.stamp = ros::Time::now();
30    static_transformStamped.header.frame_id = "baselink";
31    static_transformStamped.child_frame_id = static_turtle_name;
32    static_transformStamped.transform.translation.x = atof(argv[2]);
33    static_transformStamped.transform.translation.y = atof(argv[3]);
34    static_transformStamped.transform.translation.z = atof(argv[4]);
35    tf2::Quaternion quat;
```

上述代码在static_turtle_tf2_broadcaster.cpp 文件中

1.1.6 Supplemental information

- 发布方(ROS_wiki)：

```
$ rosrun ros_bilibili tf_static_sub
```



上述代码在static_turtle_tf2_broadcaster.cpp 文件中

1.1.6 Supplemental information

➢ 补充1：

- 当坐标系之间的相对位置固定时，所需参数也是固定的：父系坐标名称、子级坐标系名称、x偏移量、y偏移量、z偏移量、x 翻滚角度、y俯仰角度、z偏航角度
- 不同案例区别：实现逻辑相同，参数不同，ROS 系统就已经封装好了专门的节点，使用方式如下：

rosrun tf2_ros static_transform_publisher x偏移量 y偏移量 z偏移量 z偏航角度 y俯仰角度 x翻滚角度 父级坐标系 子级坐标系

- 举例：

```
$ rosrun tf2_ros static_transform_publisher 0.1 0 0.2 0 0 0 /baselink /laser
```

```
$ rosrun ros_bilibili tf_static_sub
```

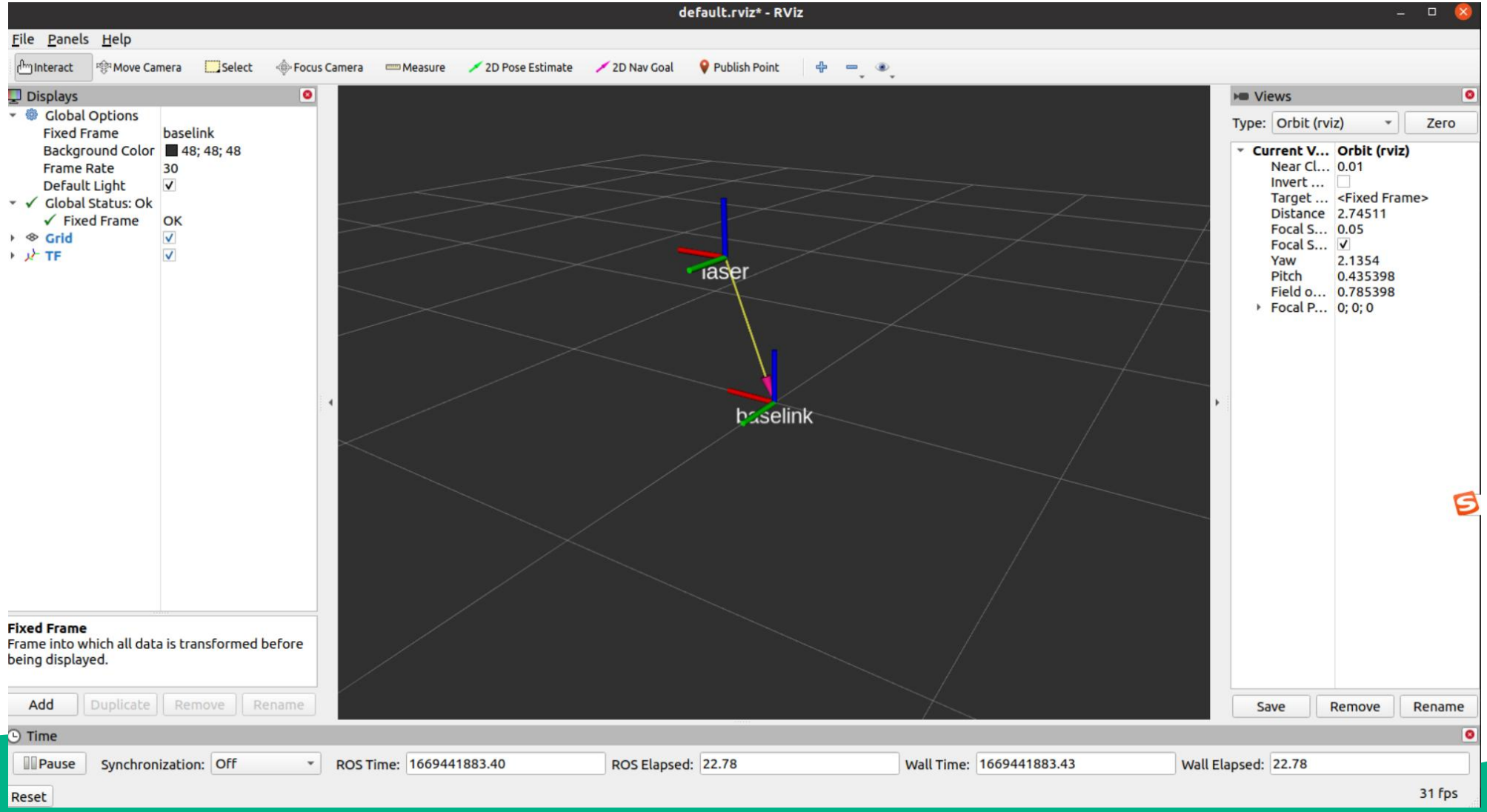
1.1.6 Supplemental information

- 补充2：
  - 可以借助 rviz 显示坐标系关系，具体操作：
    1. 新建窗口输入命令：rviz
    2. 在启动的 rviz 中设置 Fixed Frame 为 base_link
    3. 点击左下的 add 按钮，在弹出的窗口中选择 TF 组件，即可显示坐标关系

```
$ roslaunch ros_bilibili tf_static.launch
```

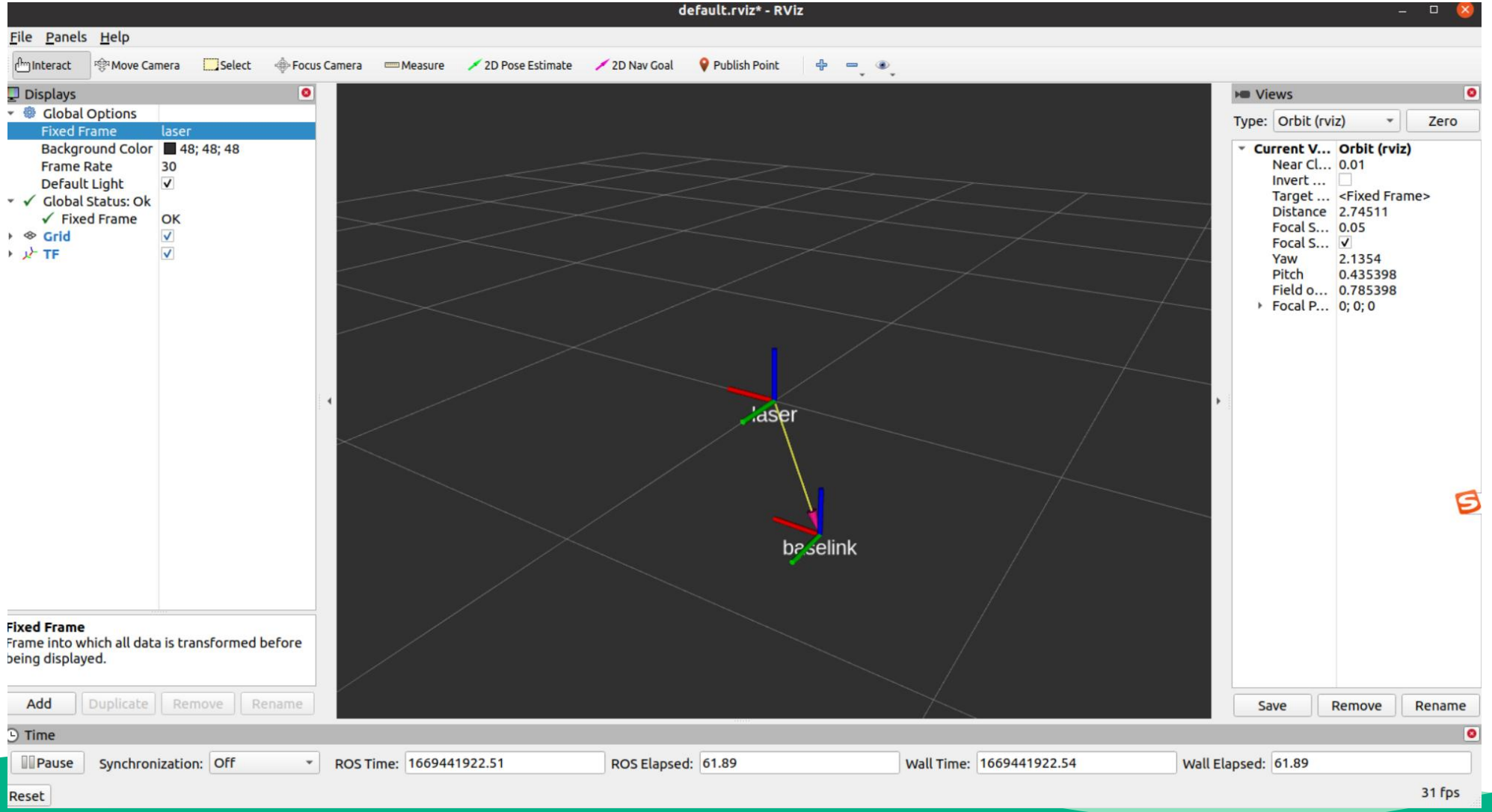
1.1 静态坐标变换

1.1.6 Supplemental information



1.1 静态坐标变换

1.1.6 Supplemental information



主观题 10分

设置

1. 坐标变换中常用的3个功能包?
2. 坐标转换中常用的2种消息类型?
3. TransformStamped和PointStamped的消息类型定义? (列出一级字段类型和字段名称)
4. ros::Time::now()和ros::Time()的含义为?
5. tf2_ros::StaticTransformBroadcaster broadcaster，broadcaster.sendTransform(ts)，这2行代码的作用是?
6. 静态坐标变换默认创建的话题是?
7. tf2_ros::Buffer buffer，tf2_ros::TransformListener listener(buffer)，这2行代码的作用是?
8. tf2_ros功能包中静态坐标变换的节点是?

正常使用主观题需2.0以上版本雨课堂

作答

Summary

1. 静态坐标变换

   a) 静态坐标变换功能包和APIs：tf2 tf2_ros tf2_geometry_msgs

   b) Tf2 messages

   c) Time and duration

   d) C++ code：

      a. 发布方：ros_bilibili ros_wiki

      b. 订阅方

   e) 补充信息：

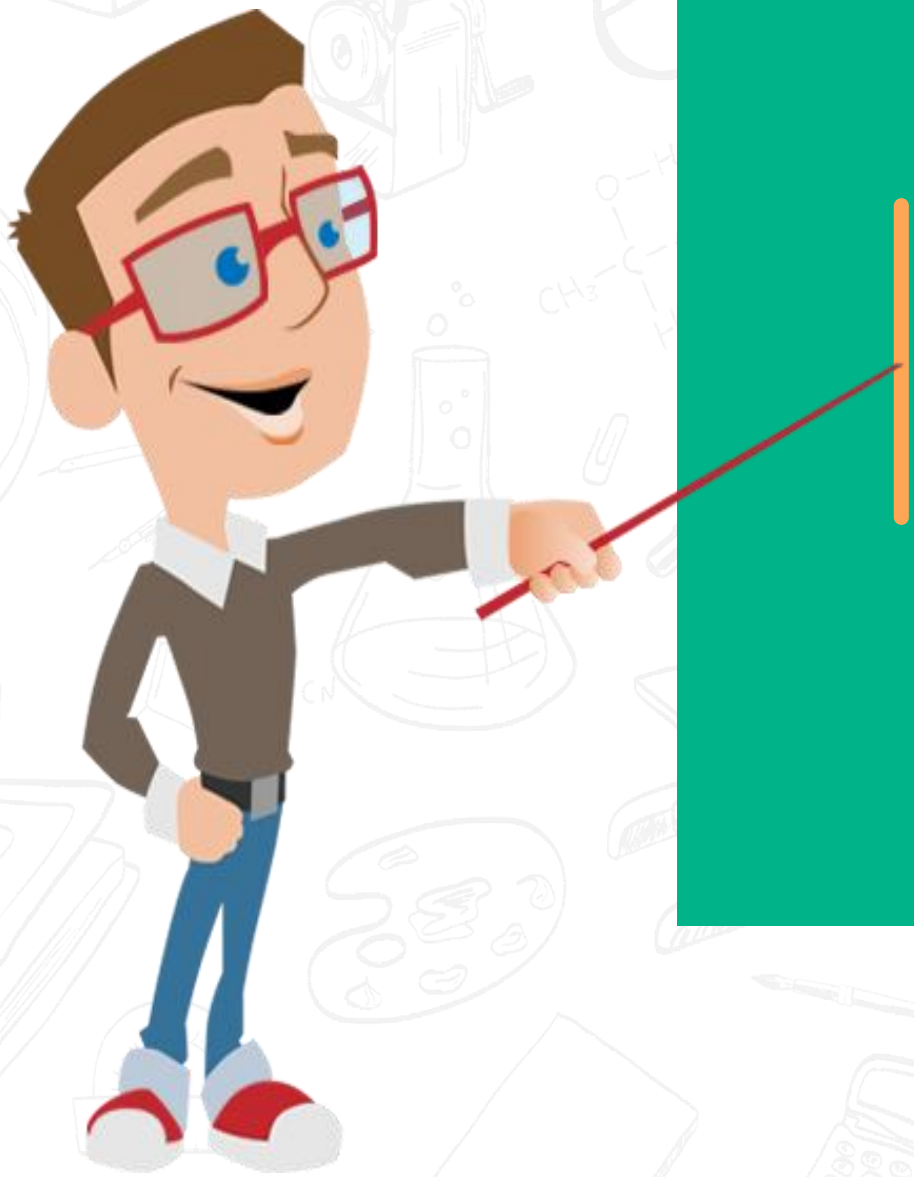
      a. static_transform_publisher

      b. rviz

Q&A

今日事今日毕

Question?



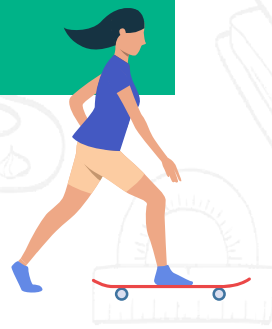
机器人控制设计实践

Robot Control Design Practice

WHERE THERE'S A WILL, THERE'S A WAY

授课教师：郄(qiè)龙飞

机器人工程



Overview lesson 12-2

1. 动态坐标变换
2. ~
3. ~
4. ~

Objectives

1、理解动态坐标变换原理及其 C++ 编程方法



Pre-assessment

1、静态坐标变换和动态坐标变换的区别?

2、动态坐标变换用到哪些函数?



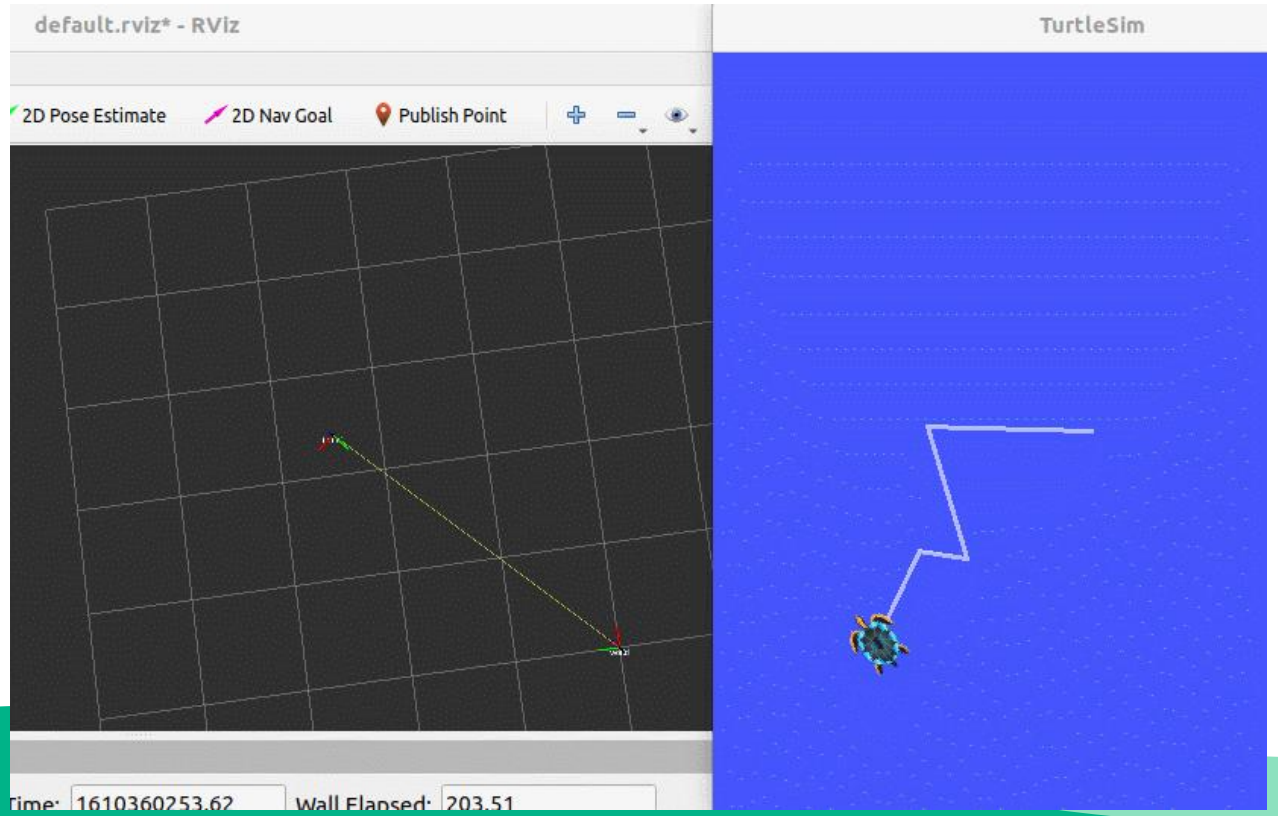
1

动态坐标变换

1.1 动态坐标变换

1.1.1 Overview

- **动态坐标变换**是指两个坐标系之间的相对位置是变化的
- **需求分析**：启动 turtlesim_node，该节点中窗体有一个世界坐标系(左下角为坐标系原点)，乌龟是另一个坐标系，键盘控制乌龟运动，将两个坐标系的相对位置动态发布



1.1 动态坐标变换

1.1.2 C++ code

- **功能实现分析：**

  1. 订阅 turtle1/pose，可以获取乌龟在世界坐标系的 x坐标、y坐标、偏移量以及线速度和角速度
  2. 将 pose 信息转换成 坐标系相对信息 并发布

- **实现流程**

  1. 新建功能包，添加依赖
  2. 创建坐标相对关系发布方(同时需要订阅乌龟位姿信息)
  3. 创建坐标相对关系订阅方
  4. 执行

1.1 动态坐标变换

1.1.2 C++ code

- 创建功能包
  - 项目功能包依赖为：roscpp rospy std_msgs tf2 tf2_ros tf2_geometry_msgs geometry_msgs turtlesim

- 创建动态坐标变换广播器：

```
#include "tf2_ros/transform_broadcaster.h"          // 包含广播器头文件
static tf2_ros::TransformBroadcaster broadcaster;   // 实例化广播器对象
```

- 创建动态坐标变换监听器：

```
#include "tf2_ros/buffer.h"                         // 包含缓存头文件
#include "tf2_ros/transform_listener.h"             // 包含监听器头文件
tf2_ros::Buffer buffer;                             // 实例化缓存对象
tf2_ros::TransformListener listener(buffer);        // 实例化监听对象
```

1.1 动态坐标变换

1.1.2 C++ code

- TF的坐标变换，表征子坐标系到父坐标系的坐标变换关系
- 通过使用TF广播，每次只需要发布相邻两个坐标系之间的坐标关系。然后通过TF监听得到这些相邻的坐标变换关系，TF就能计算出任意两个坐标系之间的坐标变换关系
- 存在问题：由于TF建立的坐标系一般都是动坐标系，也就是坐标系之间的变换关系是随着时间变化的，即存在时间对齐问题

1.1 动态坐标变换

1.1.2 C++ code

- **举例**:

1. 一个机械手臂有大臂和小臂，大臂绕着肩关节转，小臂绕着肘关节转

2. 在大臂和小臂上分别有两个IMU（A和B）可以检测姿态，A检测大臂相对肩膀的姿态，B检测小臂相对大臂的姿态。通过这两个数据计算，最后就能得到小臂相对肩膀的姿态

3. 两个IMU的数据一直在发送，如果想知道0.5S时刻小臂对肩膀的姿态，那么就需要使用0.5S时刻A的数据和0.5S时刻B的数据相乘得到，而不能使用其他时刻的数据相乘，因为需要**时间对齐**。坐标点在不同坐标系下表示的时候也是同理，所以TF中就有时间戳的概念

1.1 动态坐标变换

1.1.2 C++ code

- 静态坐标变换（发布方）：

```
// 3.创建静态坐标转换广播器
tf2_ros::StaticTransformBroadcaster broadcaster;
// 4.创建坐标系信息
geometry_msgs::TransformStamped ts;
//----设置头信息
ts.header.seq = 100;
ts.header.stamp = ros::Time::now();
ts.header.frame_id = "baselink";
//----设置子级坐标系
ts.child_frame_id = "laser";
//----设置子级相对于父级的偏移量
ts.transform.translation.x = 0.1;
ts.transform.translation.y = 0.0;
ts.transform.translation.z = 0.2;
//----设置四元数:将 欧拉角数据转换成四元数
```

这里的时间戳使用的是ros::Time::now();，也就是当前调用这条命令的时候的时间，比如得到的结果是150.3S这个时刻，表示这个tf变换是在150.3S时刻的坐标变换

1.1 动态坐标变换

1.1.2 C++ code

- 静态坐标变换（订阅方）：

```
int main(int argc, char *argv[])
{
    setlocale(LC_ALL,"");
    // 2.初始化 ROS 节点
    ros::init(argc,argv,"tf_static_sub");
    ros::NodeHandle nh;
    // 3.创建 TF 订阅节点
    tf2_ros::Buffer buffer;
    tf2_ros::TransformListener listener(buffer);
```

```
while (ros::ok())
{
// 4.生成一个坐标点(相对于子级坐标系)
    geometry_msgs::PointStamped point_laser;
    point_laser.header.frame_id = "laser";
    point_laser.header.stamp = ros::Time::now();
    point_laser.point.x = 2;
    point_laser.point.y = 3;
    point_laser.point.z = 4;
// 5.转换坐标点(相对于父级坐标系)
```

这里的时间戳使用的是ros::Time::now();，也就是当前调用这条命令的时候的时间，因为订阅方是使用buffer缓存监听tf广播，然后调用buffer.transform函数进行转换，转换时刻比150.3S这个时刻远，即有时间延迟，比如订阅者的转换时刻152.3S，即存在时间同步问题（理想情况：150.3S）

其他说明见《ROS学习之TF变换的时间戳》

1.1.2 C++ code

- 时间对齐步问题解决思路：
  - 选择TF广播中最近的时刻，使用：
    - ros::Time()或者ros::Time(0)
  - 使用这样的时间戳在进行坐标变换的时候，会使用tf广播中最近一次发布的坐标变换，认为设置的点的时间戳和变换的时间戳是时间对齐的

1.1 动态坐标变换

1.1.2 C++ code

- **发布方实现流程：**

  1.包含头文件

  2.初始化 ROS 节点

  3.创建 ROS 句柄

  4.创建订阅对象

  5.回调函数处理订阅到的数据(实现TF广播)

  5-1.创建 TF 广播器

  5-2.创建 广播的数据(通过 pose 设置)

  5-3.广播器发布数据

  6.spin

1.1.2 C++ code

- 发布方(bilibili)：

```
#include "ros/ros.h"
#include "turtlesim/Pose.h"
#include "tf2_ros/transform_broadcaster.h"
#include "geometry_msgs/TransformStamped.h"
#include "tf2/LinearMath/Quaternion.h"
```

上述代码在tf_dynamic_pub.cpp 文件中

1.1 动态坐标变换

1.1.2 C++ code

- 发布方(bilibili)：

```
void doPose(const turtlesim::Pose::ConstPtr& pose){
    static tf2_ros::TransformBroadcaster broadcaster;
    geometry_msgs::TransformStamped tf;
    tf.header.frame_id = "world";
    tf.header.stamp = ros::Time::now();
    tf.child_frame_id = "turtle1";
    tf.transform.translation.x = pose->x;
    tf.transform.translation.y = pose->y;
    tf.transform.translation.z = 0.0;
    tf2::Quaternion qtn;
    qtn.setRPY(0,0,pose->theta);
    tf.transform.rotation.x = qtn.getX();
    tf.transform.rotation.y = qtn.getY();
    tf.transform.rotation.z = qtn.getZ();
    tf.transform.rotation.w = qtn.getW();
    broadcaster.sendTransform(tf);
}
```

上述代码在tf_dynamic_pub.cpp 文件中

1.1 动态坐标变换

1.1.2 C++ code

- 发布方(bilibili)：

```
int main(int argc, char *argv[])
{
    setlocale(LC_ALL,"");
    ros::init(argc,argv,"tf_dynamic_pub");
    ros::NodeHandle nh;
    ros::Subscriber sub = nh.subscribe<turtlesim::Pose>("/turtle1/pose",1000,doPose);
    ros::spin();
    return 0;
}
```

上述代码在tf_dynamic_pub.cpp 文件中

1.1 动态坐标变换

1.1.2 C++ code

- **订阅方实现流程：**

  1.包含头文件

  2.初始化 ROS 节点

  3.创建 TF 订阅对象

  4.处理订阅的数据

  5.spinOnce()

1.1.2 C++ code

- 订阅方(bilibili):

```
#include "ros/ros.h"
#include "tf2_ros/transform_listener.h"
#include "tf2_ros/buffer.h"
#include "geometry_msgs/PointStamped.h"
#include "tf2_geometry_msgs/tf2_geometry_msgs.h"
```

上述代码在tf_dynamic_sub.cpp 文件中

1.1.2 C++ code

- 订阅方(bilibili)：

```
int main(int argc, char *argv[])
{
    setlocale(LC_ALL,"");
    ros::init(argc,argv,"tf_dynamic_sub");
    ros::NodeHandle nh;
    tf2_ros::Buffer buffer;
    tf2_ros::TransformListener listener(buffer);

    ros::Rate r(1);
    while (ros::ok())
    {
        geometry_msgs::PointStamped point_laser;
        point_laser.header.frame_id = "turtle1";
        point_laser.header.stamp = ros::Time();
```

上述代码在tf_dynamic_sub.cpp 文件中

1.1.2 C++ code

- 订阅方(bilibili)：

```
        try
        {
            geometry_msgs::PointStamped point_base;
            point_base = buffer.transform(point_laser,"world");
            ROS_INFO("坐标系%s相对于 world 的坐标为:(%.2f,%.2f,%.2f)",
point_laser.header.frame_id.c_str(),point_base.point.x,point_base.point.y,point_base.point.z);
        }
        catch(const std::exception& e)
        {
            // std::cerr << e.what() << '\n';
            ROS_INFO("程序异常:%s",e.what());
        }
        r.sleep();
        ros::spinOnce();
    }
    return 0;
}
```

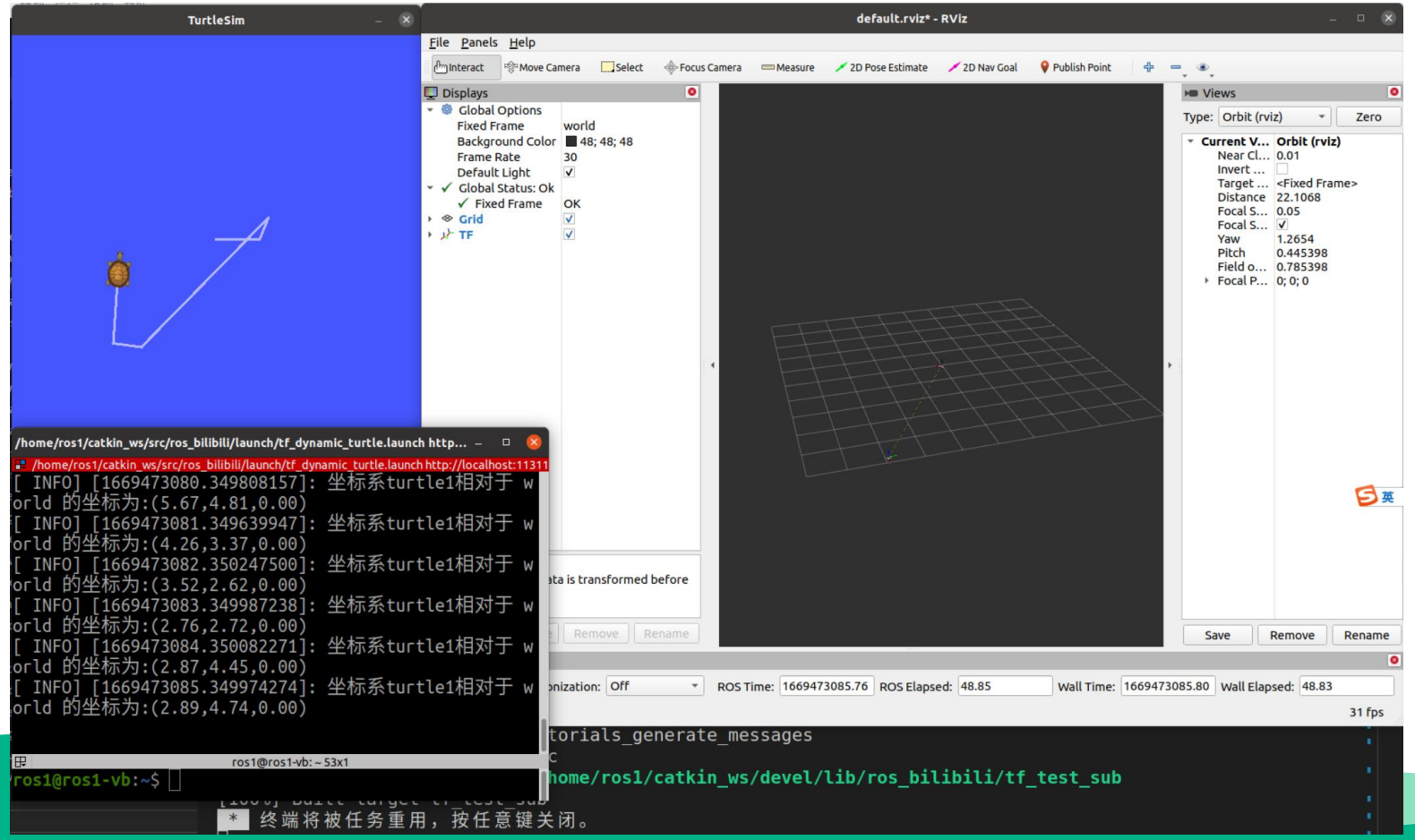
上述代码在tf_dynamic_sub.cpp 文件中

1.1 动态坐标变换

1.1.2 C++ code

- 订阅方(bilibili)：

```
$ roslaunch ros_bilibili tf_dynamic_turtle.launch
```



1.1 动态坐标变换

1.1.2 C++ code

- ROS 配置参数（补充知识）：
  - ROS参数服务器能保存的数据类型包括：字符串、整型、浮点型、布尔型、列表、词典等
  - ROS提供了两种管理参数的API，一个是在 ros::param 命名空间，一个是通过 ros::NodeHandle接口，在底层参数服务器基于XMLRPC与节点进行通信
  - 参数根据类型不同可分为：全局参数、相对参数、私有参数等。其中以“/”开头的为全局参数，以“~”开头的参数为私有参数，其他的为相对参数。
  - 在 roslaunch 文件中，当 param 作为 launch 标签的子标签出现时，参数为全局参数，当 param 标签作为 node 标签的子标签出现时，参数为相对参数

1.1.2 C++ code

- ROS 配置参数（补充知识）：
  - 通过NodeHandle检查参数：

```
ros::NodeHandle private_node("~");
if(!private_node.hasParam("turtle"))   //判断参数服务器中是否存在turtle这个参数
{


}
```

其他参数配置见《ROS之参数配置》

1.1.2 C++ code

- 发布方(roswiki)：

```
#include <ros/ros.h>
#include <tf2/LinearMath/Quaternion.h>
#include <tf2_ros/transform_broadcaster.h>
#include <geometry_msgs/TransformStamped.h>
#include <turtlesim/Pose.h>
```

上述代码在turtle_tf2_broadcaster.cpp 文件中

1.1 动态坐标变换

1.1.2 C++ code

- 发布方(roswiki)：

```
std::string turtle_name;

void poseCallback(const turtlesim::PoseConstPtr& msg){
  static tf2_ros::TransformBroadcaster br;
  geometry_msgs::TransformStamped transformStamped;
  transformStamped.header.stamp = ros::Time::now();
  transformStamped.header.frame_id = "world";
  transformStamped.child_frame_id = turtle_name;
  transformStamped.transform.translation.x = msg->x;
  transformStamped.transform.translation.y = msg->y;
  transformStamped.transform.translation.z = 0.0;
  tf2::Quaternion q;
  q.setRPY(0, 0, msg->theta);
  transformStamped.transform.rotation.x = q.x();
  transformStamped.transform.rotation.y = q.y();
  transformStamped.transform.rotation.z = q.z();
  transformStamped.transform.rotation.w = q.w();

  br.sendTransform(transformStamped);
}
```

上述代码在turtle_tf2_broadcaster.cpp 文件中

1.1 动态坐标变换

1.1.2 C++ code

- 发布方(roswiki)：

```
int main(int argc, char** argv){
  ros::init(argc, argv, "my_tf2_broadcaster");

  ros::NodeHandle private_node("~");
  if (! private_node.hasParam("turtle"))
  {
    if (argc != 2){ROS_ERROR("need turtle name as argument"); return -1;};
    turtle_name = argv[1];
  }
  else
  {
    private_node.getParam("turtle", turtle_name);
  }

  ros::NodeHandle node;
  ros::Subscriber sub = node.subscribe(turtle_name+"/pose", 10, &poseCallback);

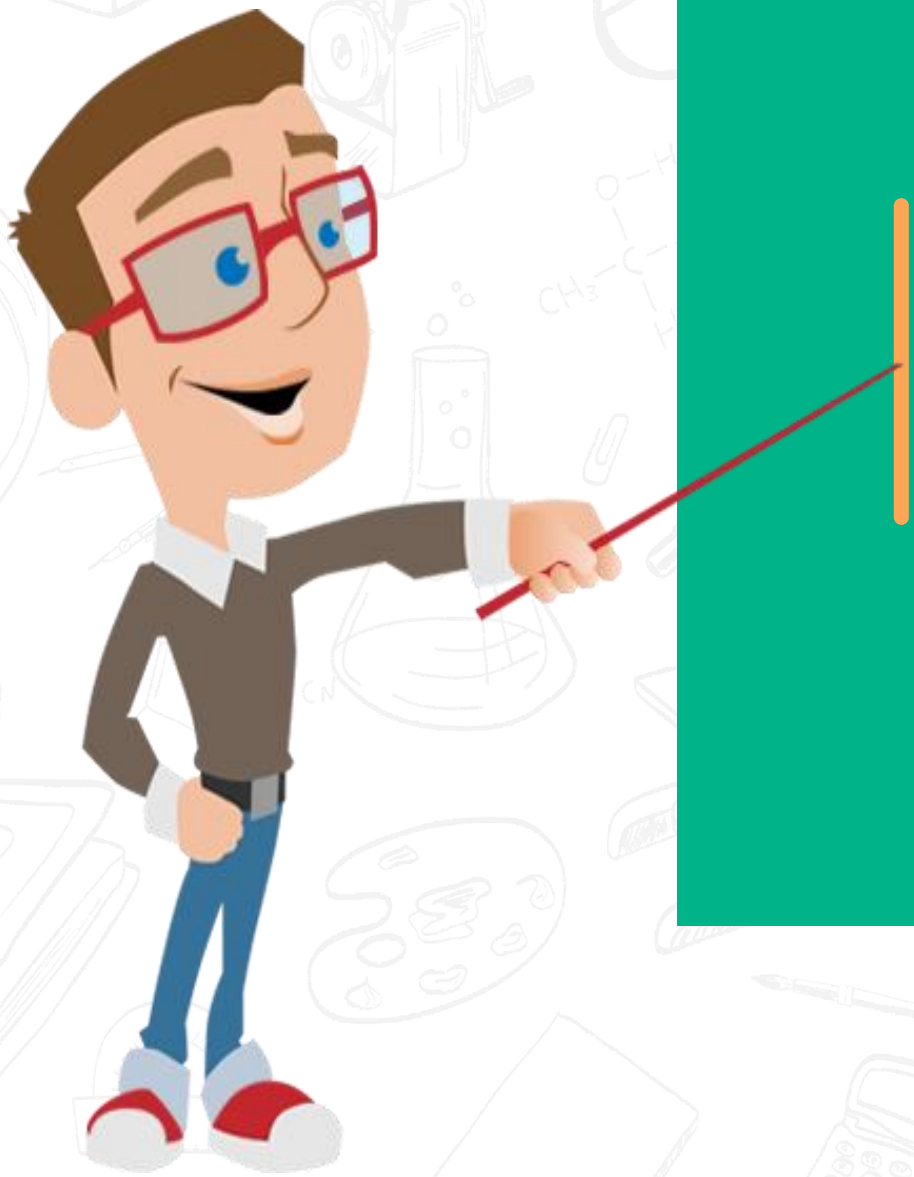
  ros::spin();
  return 0;
};
```

上述代码在turtle_tf2_broadcaster.cpp 文件中

Q&A

今日事今日毕

Question?



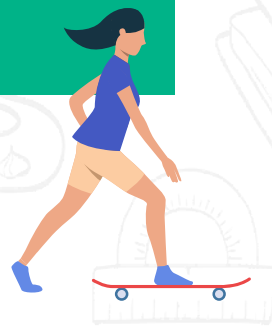
机器人控制设计实践

Robot Control Design Practice

WHERE THERE'S A WILL, THERE'S A WAY

授课教师：郄(qiè)龙飞

机器人工程



Overview lesson 12-3

1. 多坐标变换
2. Turtlesim跟随示例
3. ~
4. ~

Objectives

1、了解多坐标变换的基本原理，理解多坐标变换的 C++ 编程思想

2、了解 turtlesim 跟随示例的实现原理，理解其 C++ 代码



Pre-assessment

1、何为多坐标变换?

2、多坐标变换与静态和动态坐标变换的关系?



1

多坐标变换

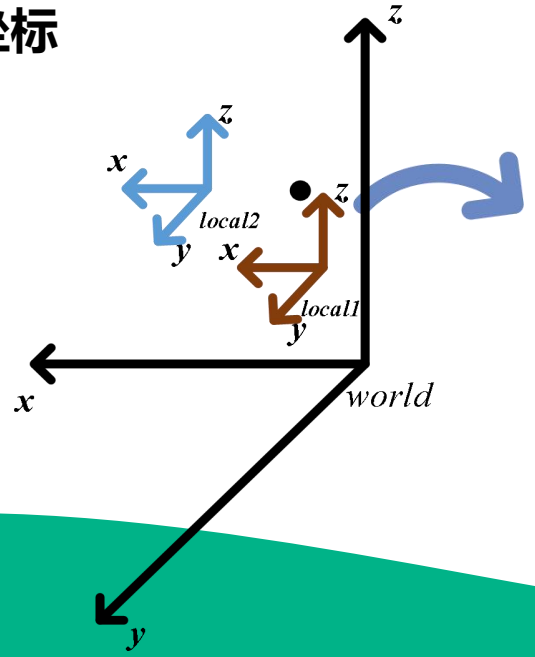
1.1 多坐标变换

1.1.1 Overview

➢ **需求描述：**

- **现有多坐标系统，父级坐标系统 world 下有两个子级系统 local1 和 local2；loacl1 相对于 world，以及 local2 相对于 world 的偏移量是分别为(1, 2, 3) 和 (4, 5, 6)**

  1. **求 local1 原点在 local2 中的坐标**
  2. **已知在 local1 中有一移动的坐标点，某一时刻该点在 local1 的坐标为(2, 3, 5)，求该时刻该点在 local2 中的坐标**



(1) 坐标为(-3, -3, -3)

(2) 坐标为(-1, 0, 2)

ROS 1.1 多坐标变换

1.1.1 Overview

➢ **需求分析：**

- 父级坐标系统 world 下有两子级系统 local1 和 local2；loacl1 相对于 world，以及 local2 相对于 world 的偏移量是分别为(1, 2, 3) 和 (4, 5, 6) → 静态坐标变换
- 求 local1原点在 local2 中的坐标 → 多坐标变换
- 已知在 local1 中有一移动的坐标点，某一时刻该点在local1的坐标为(1, 2, 3)，求该时刻该点在 local2 中的坐标 → 动态坐标变换

1.1 多坐标变换

1.1.1 Overview

- 实现分析：
  - 利用静态坐标变换，发布 local1 相对于 world，以及 local2 相对于 world 的坐标消息
  - 订阅坐标发布消息，并取出订阅的消息；利用多坐标变换，实现 local1 和 local2 的转换
  - 利用动态坐标变换，实现坐标点的转换

静态/动态坐标变换：

```
geometry_msgs::PointStamped point_world
point_world = buffer.transform(point_local1,"world")
```

多坐标变换：

```
geometry_msgs::TransformStamped tf
tf = buffer.lookupTransform("local2","local1",ros::Time(0))
```

Get the transform between two frames by frame ID assuming fixed frame.

1.1 多坐标变换

1.1.1 Overview

➢ **实现流程：**

- **新建功能包，添加依赖**
- **创建坐标相对关系发布方(需要发布两个坐标相对关系)**
- **创建坐标相对关系订阅方**
- **执行**

- **创建功能包:**
  - **创建项目功能包依赖： tf2 tf2_ros tf2_geometry_msgs roscpp rospy std_msgs geometry_msgs**

1.1.2 C++ code：方法一

- local1 发布方：

```
#include "ros/ros.h"
#include "tf2_ros/static_transform_broadcaster.h"
#include "geometry_msgs/TransformStamped.h"
#include "tf2/LinearMath/Quaternion.h"
```

1.1.2 C++ code：方法一

- local1 发布方：

```
int main(int argc, char *argv[])
{
    setlocale(LC_ALL,"");
    ros::init(argc,argv,"tf_multi_pub_local1");
    tf2_ros::StaticTransformBroadcaster broadcaster;
    geometry_msgs::TransformStamped ts;
    ts.header.seq = 1;
    ts.header.stamp = ros::Time::now();
    ts.header.frame_id = "world";
    ts.child_frame_id = "local1";
    ts.transform.translation.x = 1;
    ts.transform.translation.y = 2;
    ts.transform.translation.z = 3;
    tf2::Quaternion qtn;
    qtn.setRPY(0,0,0);
    ts.transform.rotation.x = qtn.getX();
    ts.transform.rotation.y = qtn.getY();
    ts.transform.rotation.z = qtn.getZ();
    ts.transform.rotation.w = qtn.getW();
    broadcaster.sendTransform(ts);
    ros::spin();
    return 0;
}
```

1.1.2 C++ code：方法一

- local2 发布方：

```
#include "ros/ros.h"
#include "tf2_ros/static_transform_broadcaster.h"
#include "geometry_msgs/TransformStamped.h"
#include "tf2/LinearMath/Quaternion.h"
```

1.1 多坐标变换

1.1.2 C++ code：方法一

- local2 发布方：

```
int main(int argc, char *argv[])
{
    setlocale(LC_ALL,"");
    ros::init(argc,argv,"tf_multi_pub_local2");
    tf2_ros::StaticTransformBroadcaster broadcaster;
    geometry_msgs::TransformStamped ts;
    ts.header.seq = 1;
    ts.header.stamp = ros::Time::now();
    ts.header.frame_id = "world";
    ts.child_frame_id = "local2";
    ts.transform.translation.x = 4;
    ts.transform.translation.y = 5;
    ts.transform.translation.z = 6;
    tf2::Quaternion qtn;
    qtn.setRPY(0,0,0);
    ts.transform.rotation.x = qtn.getX();
    ts.transform.rotation.y = qtn.getY();
    ts.transform.rotation.z = qtn.getZ();
    ts.transform.rotation.w = qtn.getW();
    broadcaster.sendTransform(ts);
    ros::spin();
    return 0;
}
```

1.1.2 C++ code：方法一

- local2 发布方：

```
<launch>

    <node pkg="ros_bilibili" type="tf_multi_pub_local1" name="tf_multi_pub_local1"
output="screen" />

    <node pkg="ros_bilibili" type="tf_multi_pub_local2" name="tf_multi_pub_local2"
output="screen" />


</launch>
```

1.1.2 C++ code：方法二

- local1 + local2 发布方：

```
#include "ros/ros.h"
#include "tf2_ros/static_transform_broadcaster.h"
#include "geometry_msgs/TransformStamped.h"
#include "tf2/LinearMath/Quaternion.h"
```

1.1.2 C++ code：方法二

- local1 + local2 发布方：

```
int main(int argc, char *argv[])
{
    setlocale(LC_ALL,"");
    ros::init(argc,argv,"tf_multi_pub_opti");
    tf2_ros::StaticTransformBroadcaster broadcaster;
    geometry_msgs::TransformStamped ts;
    ts.header.seq = 100;
    ts.header.stamp = ros::Time::now();
    ts.header.frame_id = "world";
    ts.child_frame_id = argv[1];
    ts.transform.translation.x = atoi(argv[2]);
    ts.transform.translation.y = atoi(argv[3]);
    ts.transform.translation.z = atoi(argv[4]);
    tf2::Quaternion qtn;
    qtn.setRPY(0,0,0);
    ts.transform.rotation.x = qtn.getX();
    ts.transform.rotation.y = qtn.getY();
    ts.transform.rotation.z = qtn.getZ();
    ts.transform.rotation.w = qtn.getW();
    broadcaster.sendTransform(ts);
    ros::spin();
    return 0;
}
```

1.1.4 C++ code：方法二

- **补充知识：**

ros传参主要有两种：第一种是通过 main 函数的 argc 和 argv 变量，第二种是通过参数服务器。具体实现如下：

(1) 通过 main 函数的 argv 变量传递参数, 在 launch 文件中使用 args 参数传递参数的值

(2) 通过参数服务器传参, 通过语句 n.param<std::string>("n_arg",n_arg,"arg1")来完成对变量 n_arg 的传参

1.1.4 C++ code：方法二

```
<launch>

    <node pkg="ros_bilibili" type="tf_multi_pub_opti" name="tf_multi_pub_opti_1"
args="local1 1 2 3" output="screen" />

    <node pkg="ros_bilibili" type="tf_multi_pub_opti" name="tf_multi_pub_opti_2"
args="local2 4 5 6" output="screen" />

</launch>
```

1.1.2 C++ code：方法三

- local1 + local2 发布方：

```
<launch>
    <node pkg="tf2_ros" type="static_transform_publisher"
name="local1" args="1 2 3 0 0 0 /world /local1" output="screen" />

    <node pkg="tf2_ros" type="static_transform_publisher"
name="local2" args="4 5 6 0 0 0 /world /local2" output="screen" />

</launch>
```

1.1 多坐标变换

1.1.2 C++ code:

- 订阅方:

```
#include "ros/ros.h"
#include "tf2_ros/transform_listener.h"
#include "tf2/LinearMath/Quaternion.h"
#include "tf2_geometry_msgs/tf2_geometry_msgs.h"
#include "geometry_msgs/TransformStamped.h"
#include "geometry_msgs/PointStamped.h"
```

1.1 多坐标变换

1.1.2 C++ code:

- 订阅方:

```
int main(int argc, char *argv[])
{   setlocale(LC_ALL,"");
    ros::init(argc,argv,"tf_multi_sub");
    ros::NodeHandle nh;
    tf2_ros::Buffer buffer;
    tf2_ros::TransformListener listener(buffer);
    ros::Rate rate(1);
    while (ros::ok())
    {
        try
        {
            geometry_msgs::TransformStamped tf;
            tf = buffer.lookupTransform("local2","local1",ros::Time(0));
            ROS_INFO("local1 相对于 local2 的坐标关系:父坐标系ID=%s",tf.header.frame_id.c_str());
            ROS_INFO("local1 相对于 local2 的坐标关系:子坐标系ID=%s",tf.child_frame_id.c_str());
            ROS_INFO("local1原点 在 local2 的坐标为:x=%.2f,y=%.2f,z=%.2f",
                    tf.transform.translation.x,
                    tf.transform.translation.y,
                    tf.transform.translation.z
                    );
```

1.1 多坐标变换

1.1.2 C++ code:

- 订阅方:

```
    geometry_msgs::PointStamped ps_local1;
    ps_local1.header.frame_id = "local1";
    ps_local1.header.stamp = ros::Time::now();
    ps_local1.point.x = 2.0;
    ps_local1.point.y = 3.0;
    ps_local1.point.z = 5.0;

    geometry_msgs::PointStamped ps_local2;
    ps_local2 = buffer.transform(ps_local1,"local2");
    ROS_INFO("local1中的点 ps_local1 在 local2 中的坐标:x=%.2f,y=%.2f,z=%.2f",
        ps_local2.point.x,
        ps_local2.point.y,
        ps_local2.point.z
        );
  }
  catch(const std::exception& e)
  {
    ROS_INFO("异常信息:%s",e.what());
  }
  rate.sleep();
  ros::spinOnce();
 }
 return 0;
}
```

1.1.2 C++ code：方法一

- 执行：

```
roslaunch ros_bilibili tf_multi_bilibili.launch
```

```
<launch>

    <node pkg="ros_bilibili" type="tf_multi_pub_local1" name="tf_multi_pub_local1"
output="screen" />

    <node pkg="ros_bilibili" type="tf_multi_pub_local2" name="tf_multi_pub_local2"
output="screen" />

    <node pkg="ros_bilibili" type="tf_multi_sub" name="tf_multi_sub" output="screen" />

    <node pkg="rviz" type="rviz" name="rviz" args="-d $(find ros_bilibili)/config/tf_multi.rviz" />

</launch>
```

1.1.2 C++ code：方法二

- 执行：

```
roslaunch ros_bilibili tf_multi_opti.launch
```

```
<launch>

    <node pkg="ros_bilibili" type="tf_multi_pub_opti" name="tf_multi_pub_opti_1"  args="local1 1 2 3" output="screen" />

    <node pkg="ros_bilibili" type="tf_multi_pub_opti" name="tf_multi_pub_opti_2"  args="local2 4 5 6" output="screen" />

    <node pkg="ros_bilibili" type="tf_multi_sub" name="tf_multi_sub" output="screen" />

    <node pkg="rviz" type="rviz" name="rviz" args="-d $(find ros_bilibili)/config/tf_multi.rviz" />

</launch>
```

1.1.2 C++ code：方法三

- 执行：

```
roslaunch ros_bilibili tf_multi_tf2ros.launch
```

```
<launch>
    <node pkg="tf2_ros" type="static_transform_publisher" name="local1" args="1 2 3 0 0 0 /world /local1" output="screen" />

    <node pkg="tf2_ros" type="static_transform_publisher" name="local2" args="4 5 6 0 0 0 /world /local2" output="screen" />

    <node pkg="ros_bilibili" type="tf_multi_sub" name="tf_multi_sub" output="screen" />

    <node pkg="rviz" type="rviz" name="rviz" args="-d $(find ros_bilibili)/config/tf_multi.rviz" />

</launch>
```



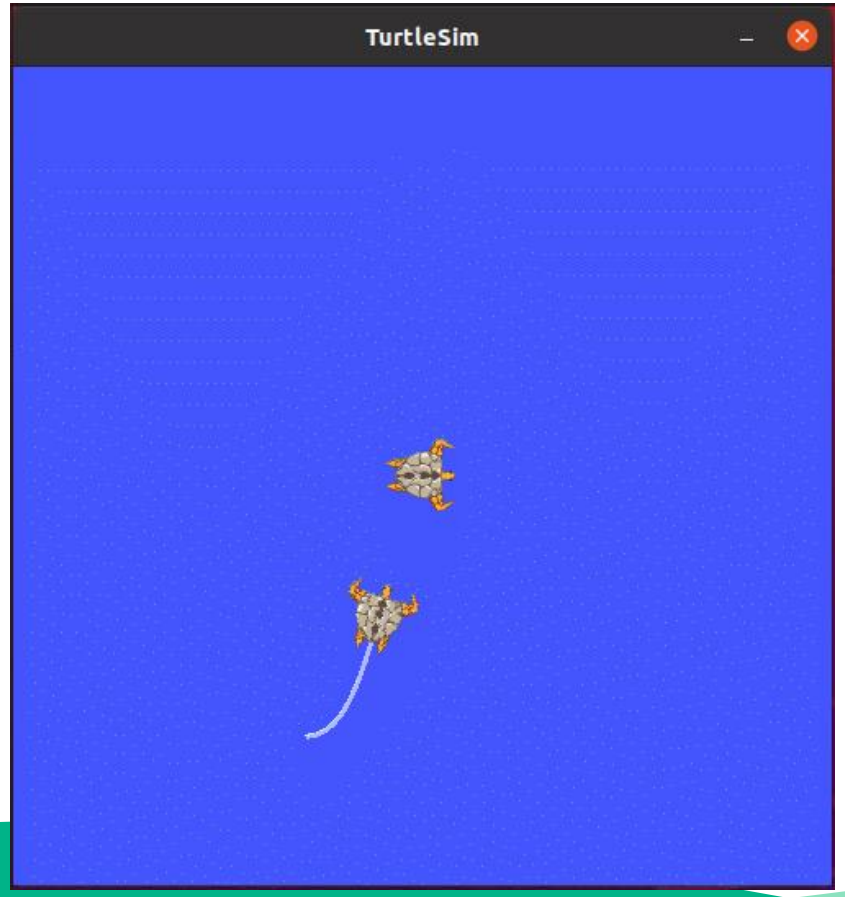
2

Turtlesim跟随示例

2.1 Turtlesim跟随示例

2.1.1 Overview

```
$ roslaunch turtle_tf2 turtle_tf2_demo.launch
```



2.1 Turtlesim跟随示例

2.1.1 Overview

- **实现分析**：
  - 乌龟跟随实现的核心，首先乌龟 A 和 B 都要发布相对世界坐标系的坐标信息；然后，订阅到该信息需要转换获取 A 相对于 B 坐标系的信息；最后，再生成速度信息，并控制 B 运动

- **执行步骤**：
  - 启动乌龟显示和控制节点 → 启动 turtlesim_node turtle_teleop_key
  - 在乌龟显示窗体中生成一只新的乌龟(需要使用服务) → 调用/spawn服务
  - 编写两只乌龟发布坐标信息的节点 → 多坐标变换（订阅/turtlex/pose x=1，2）
  - 编写订阅节点订阅坐标信息并生成新的相对关系生成速度信息 → 获取turtle2速度信息，结合turtle1和turtle2的相对位置，计算turtle2的速度信息

2.1 Turtlesim跟随示例

2.1.1 Overview

- **实现流程：**
  - 新建功能包，添加依赖
  - 编写服务客户端，用于生成一只新的乌龟
  - 编写发布方，发布两只乌龟的坐标信息
  - 编写订阅方，订阅两只乌龟信息，生成速度信息并发布
  - 执行

- **创建功能包:**
  - 创建项目功能包依赖： tf2 tf2_ros tf2_geometry_msgs roscpp rospy std_msgs geometry_msgs turtlesim

2.1 Turtlesim跟随示例

2.1.2 C++ code

- 编写服务客户端，用于生成一只新的乌龟：

```
#include "ros/ros.h"
#include "turtlesim/Spawn.h"

int main(int argc, char *argv[])
{
    setlocale(LC_ALL,"");
    ros::init(argc,argv,"create_turtle2");
    ros::NodeHandle nh;
    ros::ServiceClient client =
nh.serviceClient<turtlesim::Spawn>("/spawn");

    ros::service::waitForService("/spawn");
    turtlesim::Spawn spawn;
    spawn.request.name = "turtle2";
    spawn.request.x = 2.0;
    spawn.request.y = 2.0;
    spawn.request.theta = 1.57;
    bool flag = client.call(spawn);
    if (flag)
    {
        ROS_INFO("乌龟%s创建成
功!",spawn.response.name.c_str());
    }
    else
    {
        ROS_INFO("乌龟2创建失败!");
    }

    ros::spin();

    return 0;
}
```

2.1 Turtlesim跟随示例

2.1.2 C++ code

- 编写发布方，发布两只乌龟的坐标信息：

```
#include "ros/ros.h"
#include "turtlesim/Pose.h"
#include "tf2_ros/transform_broadcaster.h"
#include "tf2/LinearMath/Quaternion.h"
#include "geometry_msgs/TransformStamped.h"

std::string turtle_name;

void doPose(const turtlesim::Pose::ConstPtr& pose){
    static tf2_ros::TransformBroadcaster broadcaster;
    geometry_msgs::TransformStamped tf;
    tf.header.frame_id = "world";
    tf.header.stamp = ros::Time::now();
    tf.child_frame_id = turtle_name;
    tf.transform.translation.x = pose->x;
    tf.transform.translation.y = pose->y;
    tf.transform.translation.z = 0.0;
    tf2::Quaternion qtn;
    qtn.setRPY(0,0,pose->theta);
    tf.transform.rotation.x = qtn.getX();
    tf.transform.rotation.y = qtn.getY();
    tf.transform.rotation.z = qtn.getZ();
    tf.transform.rotation.w = qtn.getW();
    broadcaster.sendTransform(tf);

}
```

2.1 Turtlesim跟随示例

2.1.2 C++ code

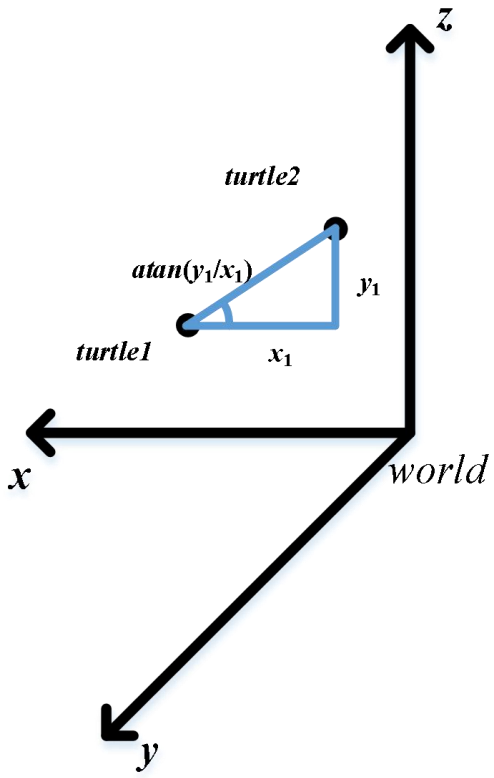
- 编写发布方，发布两只乌龟的坐标信息：

```
int main(int argc, char *argv[])
{
    setlocale(LC_ALL,"");
    ros::init(argc,argv,"tf_turtlesim_pub");
    if (argc != 2)
    {
        ROS_ERROR("请传入正确的参数");
    } else
    {
        turtle_name = argv[1];
        ROS_INFO("乌龟 %s 坐标发送启动",turtle_name.c_str());
    }
    ros::NodeHandle nh;
    ros::Subscriber sub = nh.subscribe<turtlesim::Pose>(turtle_name + "/pose",1000,doPose);
    ros::spin();
    return 0;
}
```

2.1 Turtlesim跟随示例

2.1.2 C++ code

- 编写**订阅方**，订阅两只乌龟信息，生成速度信息并发布：



- **线速度计算公式**：系数 * 距离
- **角速度计算公式**：系数 * 角度

```
ros@ROS-Course-VirtualBox:~/ROS_Course_2021_ws$ rosmsg show geometry_msgs/Twist
geometry_msgs/Vector3 linear
  float64 x
  float64 y
  float64 z
geometry_msgs/Vector3 angular
  float64 x
  float64 y
  float64 z
```

2.1 Turtlesim跟随示例

2.1.2 C++ code

- 编写订阅方，订阅两只乌龟信息，生成速度信息并发布：

```
#include "ros/ros.h"
#include "tf2_ros/transform_listener.h"
#include "geometry_msgs/TransformStamped.h"
#include "geometry_msgs/Twist.h"

int main(int argc, char *argv[])
{
    setlocale(LC_ALL,"");
    ros::init(argc,argv,"tf_turtlesim_sub");
    ros::NodeHandle nh;
    tf2_ros::Buffer buffer;
    tf2_ros::TransformListener listener(buffer);
    ros::Publisher pub = nh.advertise<geometry_msgs::Twist>("/turtle2/cmd_vel",1000);
    ros::Rate rate(10);
```

2.1 Turtlesim跟随示例

2.1.2 C++ code

- 编写订阅方，订阅两只乌龟信息，生成速度信息并发布：

```
while (ros::ok())
{
    try
    {
        geometry_msgs::TransformStamped tf = buffer.lookupTransform("turtle2","turtle1",ros::Time(0));
        geometry_msgs::Twist twist;
        twist.linear.x = 2 * sqrt(pow(tf.transform.translation.x,2) + pow(tf.transform.translation.y,2));
        twist.angular.z = 2 * atan2(tf.transform.translation.y,tf.transform.translation.x);
        pub.publish(twist);
    }
    catch(const std::exception& e)
    {
        // std::cerr << e.what() << '\n';
        ROS_INFO("tf_turtlesim_sub 错误提示:%s",e.what());
    }
    rate.sleep();
    ros::spinOnce();
}
return 0;
}
```

2.1 Turtlesim跟随示例

2.1.2 C++ code

- 执行:

```
<launch>

    <node pkg="turtlesim" type="turtlesim_node" name="turtle1" output="screen" />
    <node pkg="turtlesim" type="turtle_teleop_key" name="urtle_teleop_key" output="screen"/>

    <node pkg="ros_bilibili" type="tf_turtlesim_spawn" name="tf_turtlesim_spawn" output="screen" />

    <node pkg="ros_bilibili" type="tf_turtlesim_pub" name="tf_turtlesim_pub1" output="screen"
args="turtle1" />
    <node pkg="ros_bilibili" type="tf_turtlesim_pub" name="tf_turtlesim_pub2" output="screen"
args="turtle2" />

    <node pkg="ros_bilibili" type="tf_turtlesim_sub" name="tf_turtlesim_sub" output="screen" />

</launch>
```

2.2 Turtlesim跟随示例-roswiki

2.2.1 roslaunch

```
$ roslaunch tf2_roswiki start_demo.launch
```

```
<launch>

  <node pkg="turtlesim" type="turtlesim_node" name="turtlesim_node"/>
  <node pkg="turtlesim" type="turtle_teleop_key" name="turtle_teleop_key" output="screen"/>
  <param name="scale_linear" value="2" type="double"/>
  <param name="scale_angular" value="2" type="double"/>
  <node pkg="tf2_roswiki" type="turtle_tf2_broadcaster"
      args="/turtle1" name="turtle1_tf2_broadcaster" />
  <node pkg="tf2_roswiki" type="turtle_tf2_broadcaster"
      args="/turtle2" name="turtle2_tf2_broadcaster" />
  <node pkg="tf2_roswiki" type="turtle_tf2_listener"
      name="listener" />

</launch>
```

2.2 Turtlesim跟随示例-roswiki

2.2.1 C++ code

carrot1 与 turtle1 位置固定：

```
#include <ros/ros.h>
#include <tf2_ros/transform_broadcaster.h>
#include <tf2/LinearMath/Quaternion.h>

int main(int argc, char** argv){
  ros::init(argc, argv, "my_tf2_broadcaster");
  ros::NodeHandle node;

   tf2_ros::TransformBroadcaster tfb;
  geometry_msgs::TransformStamped
transformStamped;

  transformStamped.header.frame_id = "turtle1";
  transformStamped.child_frame_id = "carrot1";
  transformStamped.transform.translation.x = 0.0;
  transformStamped.transform.translation.y = 2.0;
  transformStamped.transform.translation.z = 0.0;
  };
```

```
tf2::Quaternion q;
       q.setRPY(0, 0, 0);
  transformStamped.transform.rotation.x = q.x();
  transformStamped.transform.rotation.y = q.y();
  transformStamped.transform.rotation.z = q.z();
  transformStamped.transform.rotation.w = q.w();

  ros::Rate rate(10.0);
  while (node.ok()){
    transformStamped.header.stamp = ros::Time::now();
    tfb.sendTransform(transformStamped);
    rate.sleep();
    printf("sending\n");
  }
```

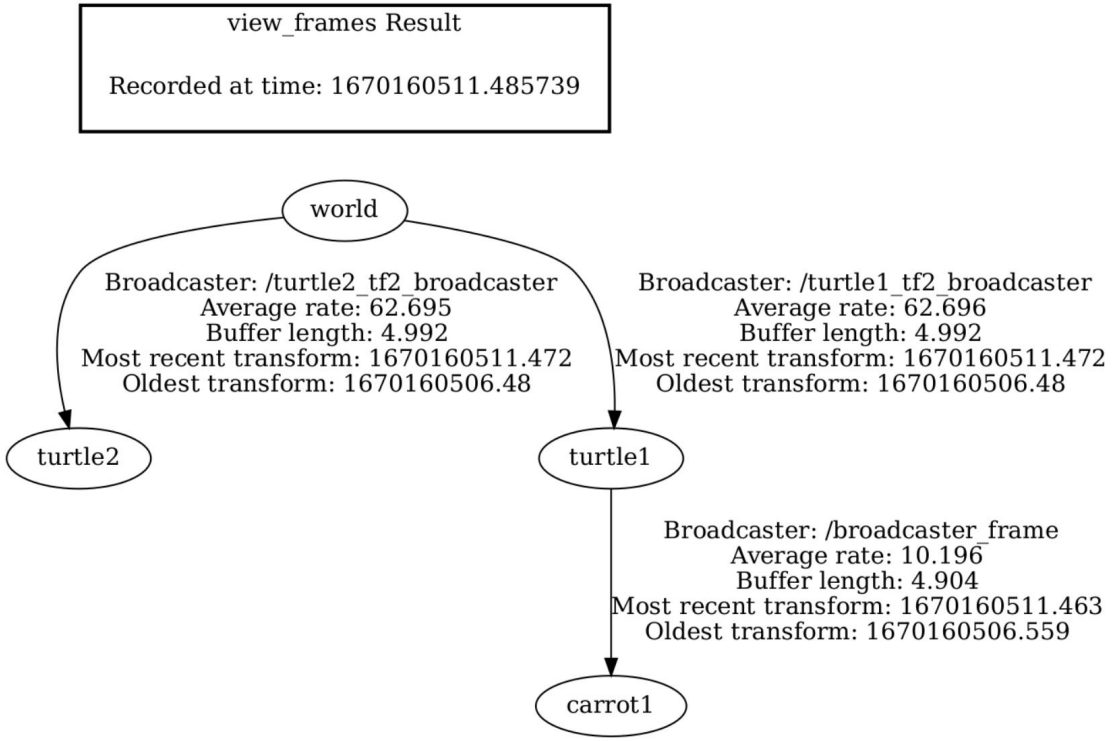
frame_tf2_broadcaster.cpp

2.2 Turtlesim跟随示例-roswiki

2.2.1 C++ code

```
rosrun tf2_tools view_frames.py
```

```
evince frames.pdf
```



2.2 Turtlesim跟随示例-roswiki

2.2.1 C++ code

carrot1 与 turtle1 位置固定：

```
// transformStamped = tfBuffer.lookupTransform("turtle2", "turtle1",
//                                ros::Time(0));

transformStamped = tfBuffer.lookupTransform("turtle2", "carrot1",
                                ros::Time(0));
```

turtle_tf2_listener.cpp

2.2 Turtlesim跟随示例-roswiki

2.2.1 roslaunch

```
$ roslaunch tf2_roswiki start_demo_update0.launch
```

```
<launch>

  <node pkg="turtlesim" type="turtlesim_node" name="turtlesim_node"/>
  <node pkg="turtlesim" type="turtle_teleop_key" name="turtle_teleop_key" output="screen"/>
  <param name="scale_linear" value="2" type="double"/>
  <param name="scale_angular" value="2" type="double"/>
  <node pkg="tf2_roswiki" type="turtle_tf2_broadcaster"
      args="/turtle1" name="turtle1_tf2_broadcaster" />
  <node pkg="tf2_roswiki" type="turtle_tf2_broadcaster"
      args="/turtle2" name="turtle2_tf2_broadcaster" />
  <node pkg="tf2_roswiki" type="turtle_tf2_listener"
      name="listener" />
  <node pkg="tf2_roswiki" type="frame_tf2_broadcaster"
      name="broadcaster_frame" />

</launch>
```

turtle2 跟随 turtle1，无变化

2.2 Turtlesim跟随示例-roswiki

2.2.1 roslaunch

```
$ roslaunch tf2_roswiki start_demo_update1.launch
```

```
<launch>
  <node pkg="turtlesim" type="turtlesim_node" name="turtlesim_node"/>
  <node pkg="turtlesim" type="turtle_teleop_key" name="turtle_teleop_key" output="screen"/>
  <param name="scale_linear" value="2" type="double"/>
  <param name="scale_angular" value="2" type="double"/>
  <node pkg="tf2_roswiki" type="turtle_tf2_broadcaster"
        args="/turtle1" name="turtle1_tf2_broadcaster" />
  <node pkg="tf2_roswiki" type="turtle_tf2_broadcaster"
        args="/turtle2" name="turtle2_tf2_broadcaster" />
     <!-- update: change urtle_tf2_listener to turtle_tf2_listener_update1:
      change turtle1 to carrot1  -->
  <node pkg="tf2_roswiki" type="turtle_tf2_listener_update1"
        name="listener" />
  <node pkg="tf2_roswiki" type="frame_tf2_broadcaster"
        name="broadcaster_frame" />
</launch>
```

turtle2 跟随 turtle1，y方向相差2（carrot1 与 turtle1 y方向相差2）

2.2 Turtlesim跟随示例-roswiki

2.2.1 C++ code

carrot1 与 turtle1 位置随时间变化：

```
#include <ros/ros.h>
#include <tf2_ros/transform_broadcaster.h>
#include <tf2/LinearMath/Quaternion.h>

int main(int argc, char** argv){
  ros::init(argc, argv, "my_tf2_broadcaster");
  ros::NodeHandle node;

   tf2_ros::TransformBroadcaster tfb;
  geometry_msgs::TransformStamped
transformStamped;

  transformStamped.header.frame_id = "turtle1";
  transformStamped.child_frame_id = "carrot1";
  transformStamped.transform.translation.z = 0.0;
  };
```

```
tf2::Quaternion q;
        q.setRPY(0, 0, 0);
  transformStamped.transform.rotation.x = q.x();
  transformStamped.transform.rotation.y = q.y();
  transformStamped.transform.rotation.z = q.z();
  transformStamped.transform.rotation.w = q.w();

  ros::Rate rate(10.0);
  while (node.ok()){
    transformStamped.header.stamp = ros::Time::now();
    transformStamped.transform.translation.x =
2.0*sin(ros::Time::now().toSec());
    transformStamped.transform.translation.y =
2.0*cos(ros::Time::now().toSec());
    tfb.sendTransform(transformStamped);
    rate.sleep();
    printf("sending\n");
  }
```

frame_tf2_broadcaster.cpp

2.2 Turtlesim跟随示例-roswiki

2.2.1 roslaunch

```
$ roslaunch tf2_roswiki start_demo_update2.launch
```

```
<launch>
  <node pkg="turtlesim" type="turtlesim_node" name="turtlesim_node"/>
  <node pkg="turtlesim" type="turtle_teleop_key" name="turtle_teleop_key" output="screen"/>
  <param name="scale_linear" value="2" type="double"/>
  <param name="scale_angular" value="2" type="double"/>
  <node pkg="tf2_roswiki" type="turtle_tf2_broadcaster"
      args="/turtle1" name="turtle1_tf2_broadcaster" />
  <node pkg="tf2_roswiki" type="turtle_tf2_broadcaster"
      args="/turtle2" name="turtle2_tf2_broadcaster" />
  <!-- update: change urtle_tf2_listener to turtle_tf2_listener_update1 -->
  <node pkg="tf2_roswiki" type="turtle_tf2_listener_update1"
      name="listener" />
    <!-- update: change frame_tf2_broadcaster to frame_tf2_broadcaster_update2 -->
  <node pkg="tf2_roswiki" type="frame_tf2_broadcaster_update2"
      name="broadcaster_frame" />
</launch>
```

turtle2 跟随 turtle1，动作随时间变化

2.2 Turtlesim跟随示例-roswiki

2.2.1 roslaunch

```
$ roslaunch tf2_roswiki start_demo_update3.launch
```

```
<launch>
  <node pkg="turtlesim" type="turtlesim_node" name="turtlesim_node"/>
  <node pkg="turtlesim" type="turtle_teleop_key" name="turtle_teleop_key" output="screen"/>
  <param name="scale_linear" value="2" type="double"/>
  <param name="scale_angular" value="2" type="double"/>
  <node pkg="tf2_roswiki" type="turtle_tf2_broadcaster"
      args="/turtle1" name="turtle1_tf2_broadcaster" />
  <node pkg="tf2_roswiki" type="turtle_tf2_broadcaster"
      args="/turtle2" name="turtle2_tf2_broadcaster" />

  <!-- update: change urtle_tf2_listener to turtle_tf2_listener_update2 -->
  <node pkg="tf2_roswiki" type="turtle_tf2_listener_update2"
      name="listener" />

</launch>
```

turtle2 跟随 5秒前的turtle1，turtle2无序运动
（相当于5秒前的turtle2 跟随 5秒前的turtle1）

2.2 Turtlesim跟随示例-roswiki

2.2.1 roslaunch

```
$ roslaunch tf2_roswiki start_demo_update4.launch
```

```
<launch>
  <node pkg="turtlesim" type="turtlesim_node" name="turtlesim_node"/>
  <node pkg="turtlesim" type="turtle_teleop_key" name="turtle_teleop_key" output="screen"/>
  <param name="scale_linear" value="2" type="double"/>
  <param name="scale_angular" value="2" type="double"/>
  <node pkg="tf2_roswiki" type="turtle_tf2_broadcaster"
      args="/turtle1" name="turtle1_tf2_broadcaster" />
  <node pkg="tf2_roswiki" type="turtle_tf2_broadcaster"
      args="/turtle2" name="turtle2_tf2_broadcaster" />

  <!-- update: change urtle_tf2_listener to turtle_tf2_listener_update3 -->
  <node pkg="tf2_roswiki" type="turtle_tf2_listener_update3"
      name="listener" />

</launch>
```

turtle2 跟随 5秒前的turtle1，turtle2无序运动
（相当于5秒前的turtle2 跟随 现在的turtle1）

tip：默认保留10s的坐标变换

Summary

1. 动态坐标变换

   a) 动态坐标变换功能包：tf2 tf2_ros tf2_geometry_msgs

   b) C++ code（三种方法）

2. Turtlesim跟随示例

   a) 实现分析

   b) 执行步骤

   c) 实现流程

   d) C++ code：turtle2 速度计算

Q&A

今日事今日毕

Question?